

LEARNING DEXTEROUS MANIPULATION FROM ROBUST REWARD MODELING

by

JACOB KRUSE

(Under the Direction of Prashant Doshi)

ABSTRACT

Reinforcement learning (RL) has shown strong performance in simulation, but applying RL to real-world tasks remains challenging due to environmental uncertainty, high-dimensionality, data collection, and safety considerations. Incorporating strategies from robust, offline, and model-based RL, this work proposes a policy learning framework for contextual bandit settings to tackle the many challenges of real-world RL. An initial data collection strategy is implemented to promote sample diversity. An ensemble of reward networks are trained on the data and used to predict the reward and to penalize uncertain estimates. A new objective called the Perturbed Action Robustness (PAR) objective is introduced, encouraging selection of actions that achieve high reward and remain stable under perturbations. The method further incorporates iterative deployment and data aggregation to continuously refine the reward model and policy. Experimental results in a custom gym environment and a robotic grasping task demonstrate improved performance compared to baseline methods.

INDEX WORDS: Robust reinforcement learning, Model-based reinforcement learning, Offline reinforcement learning, Robotic grasping, Regularization

LEARNING DEXTEROUS MANIPULATION FROM ROBUST REWARD MODELING

by

JACOB KRUSE

Bachelor of Science, Western Illinois University, 2024

A Thesis Submitted to the Graduate Faculty of the
University of Georgia in Partial Fulfillment of the Requirements for the Degree.

MASTER OF SCIENCE

ATHENS, GEORGIA

2026

©2026

Jacob Kruse

All Rights Reserved

LEARNING DEXTEROUS MANIPULATION FROM ROBUST REWARD MODELING

by

JACOB KRUSE

Major Professor: Prashant Doshi

Committee: Mehmet Kurum
Jin Lu

Electronic Version Approved:

Ron Walcott

Dean of the Graduate School

The University of Georgia

June 2026

DEDICATION

To mom and dad, this would not have been possible without your support.

ACKNOWLEDGMENTS

First and foremost, I want to thank everyone who has supported me in some way up until this point. Although I do not have enough room to mention you all, I could not have done it without you.

Secondly, I want to thank my parents. I would not be the person I am today without your love and support. For something as unexpected and crazy as moving to Georgia weeks before the semester began, your approval and help was something that came without question. No matter the circumstance, you will always support and be there for me.

Next, I would like to give thanks to all of my amazing friends here in Athens. Matt, Matt, and Allison, you are some of the best people that I have ever met, and it is hard to think that we began as strangers who met on Facebook. You guys made my time here enjoyable and gave me an escape from my work.

I would also like to express my gratitude for Dr. Shin. I appreciate your amazing mentorship during my time as an electrical engineer, pushing me to be the best no matter what. You are also a big reason why I am here today, guiding me during my pursuit of a Masters.

And finally, I would like to thank all of the committee. Dr. Doshi, Dr. Kurum, and Dr. Lu, I appreciate the time you all spent guiding me in my research.

CONTENTS

Dedication	iv
Acknowledgments	v
List of Figures	viii
List of Tables	ix
List of Algorithms	x
1 Introduction	1
1.1 Contributions	3
1.2 Thesis Structure	4
2 Related Work	6
2.1 Learning Success Models for Grasping	6
2.2 Robust Reinforcement Learning	7
2.3 Offline Reinforcement Learning	9
2.4 Model-Based Reinforcement Learning	10
3 Foundational Concepts	12
3.1 Contextual Bandits	12
3.2 Policy Gradient Reinforcement Learning	14

3.3	Reward Modeling	18
3.4	Regularization in Offline Reinforcement Learning	21
4	Methodology	23
4.1	Approximating an Accurate Reward Model	23
4.1.1	Initial Data Collection Strategy	24
4.1.2	Leveraging Ensemble Networks	25
4.2	Improved Policy Optimization	27
4.2.1	Perturbed Action Robustness Objective	28
4.2.2	Iterative Deployment of Trained Policy	30
5	Experiments	33
5.1	Simulation Experimental Setup	33
5.2	Physical Experimental Setup	34
5.2.1	Robotic Task Description	35
5.2.2	Context Estimation	36
5.2.3	Context Definition	38
5.2.4	Action Definition	40
5.2.5	Reward Definition	42
5.2.6	Neural Network Architectures	43
5.3	Baselines	43
5.4	Experiments and Results	45
5.4.1	Objective Comparison in Simulation	45
5.4.2	Sensitivity Analysis in Simulation	49
5.4.3	Objective Comparison on Robotic Task	50
6	Conclusion	53
6.1	Limitations	54

6.2 Future Work	55
Appendices	56
A Theoretical Justification for Uncertainty Penalization	56
B Theoretical Analysis of Convergence	58
C Theoretical Analysis of Iterative Deployment Stability	62
Bibliography	66

LIST OF FIGURES

1.1	PAR Function.	3
2.1	Offline Reinforcement Learning.	10
2.2	Model-Based Reinforcement Learning.	11
4.1	Proposed Policy Learning Framework.	31
5.1	Buffalo Gym Expected Reward Functions.	34
5.2	Physical Experimental Setup.	36
5.3	Robot Trajectory.	36
5.4	Context Estimation.	37
5.5	Discretized Workspace.	40
5.6	Local Coordinate Frame.	42
5.7	Convergence Examples from Objective Comparison.	48
5.8	Function 1 Sensitivity Analysis.	49
5.9	Function 4 Sensitivity Analysis.	50
5.10	Function 5 Sensitivity Analysis.	50

LIST OF TABLES

5.1	Maximum Action Perturbations.	34
5.2	Objective Comparison Results for Function 1.	46
5.3	Objective Comparison Results for Function 2.	46
5.4	Objective Comparison Results for Function 3.	47
5.5	Objective Comparison Results for Function 4.	47
5.6	Objective Comparison Results for Function 5.	47
5.7	Function 1 Sensitivity Analysis Results.	49
5.8	Function 4 Sensitivity Analysis Results.	50
5.9	Function 5 Sensitivity Analysis Results.	50
5.10	Grasp Success Rates from Robotic Sorting Task.	51
5.11	Complete Grasp Success Rates for Actions without Noise.	52
5.12	Complete Grasp Success Rates for Actions with Noise.	52
5.13	Complete Grasp Success Rates for All Actions.	52

LIST OF ALGORITHMS

1	Train Expected Reward Ensemble	27
2	Policy Optimization	32
3	Sorting	35

CHAPTER I

INTRODUCTION

Reinforcement learning (RL) has achieved remarkable success in simulation, where agents can execute many actions and easily evaluate rewards. In simulated environments, large amounts of data can be generated quickly and safely, allowing policies to improve through extensive trial and error. For instance, Vecerik et al., 2017 trains a robotic manipulator in simulation to perform a variety of manipulation tasks, while Mnih et al., 2015 uses RL to train an agent to play Atari games. These methods demonstrate the ability of RL algorithms to learn complex behaviors when there are large amounts of interaction data available to the agent.

Recently, RL has been increasingly applied to real-world tasks, motivated by the desire to tackle challenging problems beyond simulation. For example, Smith et al., 2022 demonstrates how a legged robot can learn to traverse real world terrains, and Wu et al., 2023 shows the application of their method to multiple physical platforms, including a quadruped robot, a robotic manipulator, and Sphero robot. Another prominent application of real-world RL is robotic grasping (Han et al., 2023), due to its widespread relevance in manipulation tasks such as warehouse automation and manufacturing. In these domains, robust grasping strategies are essential for practical deployment.

Despite this progress, the application of RL in the real world remains significantly more difficult than in simulation. The use of physical systems introduces uncertainty in sensing and actuation, while environmental factors introduce variability that further complicates learning. Additionally, robotic tasks

often involve high-dimensional continuous state and action spaces, leading to challenges associated with the curse of dimensionality. Data collection is also costly and time-consuming, as it requires executing actions on physical hardware. Furthermore, safety must be considered to avoid damaging equipment or causing injury. These challenges make it difficult to apply traditional RL methods, which typically rely on large amounts of accurate data obtained through online interaction.

Offline reinforcement learning is a popular strategy for addressing these limitations in physical systems. Offline RL methods learn policies entirely from pre-collected datasets without additional environmental interaction during training. This greatly improves sample efficiency by enabling the algorithm to focus entirely on policy optimization rather than exploration. However, offline RL introduces its own challenges, most notably distribution shift. Since the learned policy may select actions that differ from those present in the dataset, the value estimates for these actions cannot be evaluated reliably.

To mitigate this issue, many offline RL approaches incorporate techniques such as regularization and policy constraints. Regularization involves adding a term to the learning objective that encourages desirable behaviors in the learned policy, such as safer actions, improved exploration, or policy outputs that remain close to the collected data distribution. Incentivizing these behaviors can help stabilize training and reduce the risk of exploiting inaccuracies in value estimates.

Similarly, maximum entropy reinforcement learning (MaxEnt RL) (Haarnoja et al., 2017) alters the standard RL objective by incorporating an entropy term. This encourages the policy to maximize both expected return and entropy, promoting stochasticity in the policy and encouraging broader action distributions. While this resembles regularization with the additional term, MaxEnt RL fundamentally redefines the optimization objective. A well-known example is Soft Actor-Critic (SAC) (Haarnoja et al., 2018), which optimizes the MaxEnt objective and has demonstrated strong performance in continuous control tasks.

Inspired by these approaches and with real-world deployment in mind, this work introduces a policy learning framework for contextual bandits centered around a novel robustness measure called the Perturbed Action Robustness (PAR) function. The PAR function measures an action’s robustness to noise

and is incorporated into the RL objective, encouraging the selection of actions that are both high reward and land in high reward regions. An illustration of the PAR function is provided in Figure 1.1. To optimize this objective, an approximate reward model is learned using an initial data collection strategy designed to promote diversity and broad input coverage. This learned model is represented by an ensemble of networks to improve reward estimation and to penalize uncertain predictions. Finally, after a policy is trained with the PAR objective, the policy is repeatedly deployed to collect on-policy data, which is aggregated to iteratively improve both the reward model and policy. The approach is evaluated in simulation using a custom Gym environment and on a physical robotic manipulation task.

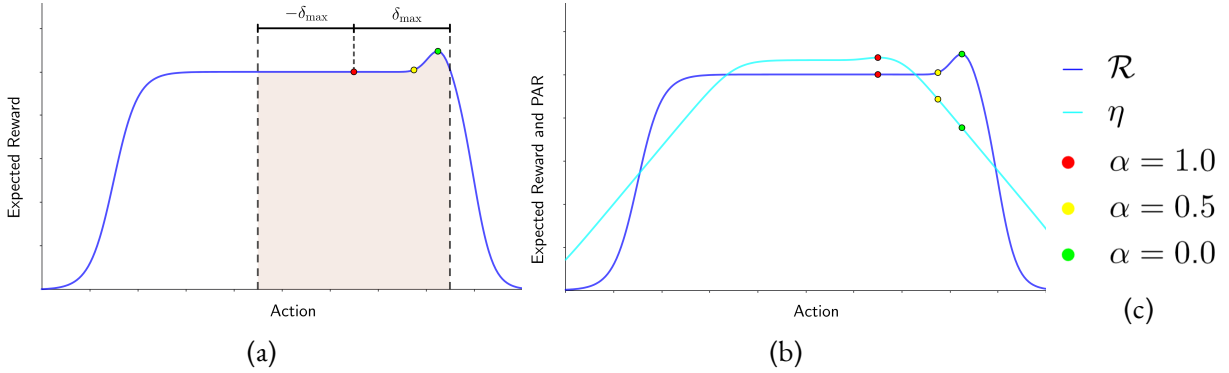


Figure 1.1: PAR Function. For continuous action spaces, the Perturbed Action Robustness (PAR) function η measures the average reward in a region bounded by the maximum action perturbation $\delta_{\max}$. The PAR objective $((1 - \alpha)\mathcal{R} + \alpha\eta)$ trades off performance/reward ($\mathcal{R}$) for robustness/average reward (η) via the robustness coefficient α . When $\alpha = 0$, the policy prioritizes the highest reward; when $\alpha = 1$, it prioritizes the highest average reward. Fig. 1.1a illustrates the PAR integration region and Fig. 1.1b compares the $\mathcal{R}$ and η functions, also showing the preferred actions for differing values of α .

1.1 Contributions

Motivated by the challenges of real-world RL and inspired by regularization-based approaches, this work makes the following contributions:

- A robustness measure, the Perturbed Action Robustness (PAR) function, which is incorporated into the standard RL learning objective to produce the PAR objective, encouraging selection of actions that are both high-reward and robust to perturbations.

- A policy learning framework that:
 - Uses an initial data collection strategy to produce diverse yet sample-efficient datasets.
 - Learns an ensemble of expected reward functions to quantify and penalize uncertainty.
 - Iteratively trains and deploys a policy optimized with the proposed robust objective to refine both the reward model and the policy.
- Evaluation of the proposed method in a custom Gym environment and a physical robotic sorting task, demonstrating improved performance compared to baseline methods.

1.2 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2 – Related Work:** This chapter reviews prior work on learning grasp success models, robust reinforcement learning, offline reinforcement learning, and model-based reinforcement learning. While these areas share similarities with the proposed method, they differ from the approach presented in this work.
- **Chapter 3 – Foundational Concepts:** This chapter introduces the foundational concepts necessary to understand the proposed method. The contextual bandit framework, policy gradient reinforcement learning, reward modeling, and regularization are presented to provide the necessary background.
- **Chapter 4 – Methodology:** This chapter presents the proposed method in detail. First, it describes strategies for learning an accurate reward model, including an effective data collection strategy and the use of ensemble networks to capture uncertainty. Then, it introduces the main algorithm for policy learning, which utilizes the newly defined PAR objective and an iterative deployment strategy to progressively improve the policy.

- **Chapter 5 – Experiments:** This chapter describes the experimental evaluation of the proposed method. It includes details of the simulation setup, robotic grasping setup, baselines, and results demonstrating the robustness of the proposed approach.
- **Chapter 6 – Conclusion:** This final chapter summarizes the contributions of the thesis, discusses the implications of the results, and outlines possible directions for future work.

CHAPTER 2

RELATED WORK

This chapter reviews the key areas of prior work that inspired the proposed approach. Section 2.1 discusses learning-based methods for robotic grasping, with a focus on models that predict grasp success from sensory input. Next, Section 2.2 introduces robust reinforcement learning, which addresses challenges arising from uncertainty and adversarial effects. Section 2.3 describes offline reinforcement learning, emphasizing learning from static datasets. Finally, Section 2.4 explains model-based reinforcement learning, highlighting techniques that leverage learned models. These areas provided the conceptual foundation and ideas that motivated the development of this work.

2.1 Learning Success Models for Grasping

Robotic grasping is a major area of study within robotics due to its wide use in manipulation tasks. As these tasks become more complex, traditional hand-engineered approaches are becoming increasingly ineffective. As a result, a substantial portion of research in robotic grasping has shifted toward learning-based methods that leverage datasets to model grasp outcomes. Many of these approaches focus on learning a grasp success model that predicts whether a proposed grasp action will succeed in the current state of the environment.

These models are trained to predict the success probability of candidate grasps using information such as RGB images, depth maps, object positions, or affordance representations. For example, after collecting data from 50,000 grasp attempts, Pinto and Gupta, 2016 trained a Convolutional Neural Network that takes image patches as input and outputs the probability of grip success for a set of candidate grasp angles. Similarly, Levine et al., 2018 learned a grasp prediction network that predicts the likelihood of a successful grasp from visual input and integrated this estimate within a continuous visual servoing controller.

Other work has explored the use of synthetic datasets to train these grasp evaluation models. After generating a large dataset of synthetic point clouds, Mahler et al., 2017 learned a grip robustness function that evaluates candidate grasps and enables planning by selecting the highest-scoring action. This approach demonstrates how simulated data can be used to train these models while allowing them to generalize to real world-tasks. Other methods, such as Asif et al., 2019 and Detry et al., 2011, learn these grasp success models from interaction data and utilize them for simple evaluation or to find high success actions.

The method in this paper follows this line of work by learning a grasp success model that maps states and actions to success probability. However, rather than using this learned model solely as an evaluation metric for grasp selection, the success predictor is interpreted as the expected reward function in this work. This reward model is then incorporated into the reinforcement learning framework to optimize an effective grasping policy.

2.2 Robust Reinforcement Learning

Robust reinforcement learning (Moos et al., 2022) encompasses a class of methods that extend the standard reinforcement learning objective to account for hostile environmental effects or adversarial factors. These approaches are especially useful for real world applications, where uncertainty and disturbances are inherent to the environment. The goal therefore is to learn a policy that achieves high expected reward while remaining effective in the presence of noise, adversaries, and uncertainty.

Prior work in robust RL can be categorized based on the source of uncertainty that the method aims to address. The most common approaches focus on robustness with respect to uncertainty in the

transition dynamics or reward function. For example, Bagnell et al., 2001 accounts for the stochastic nature of the environment by reformulating the reinforcement learning objective as a minimax problem, where the policy is optimized against the worst-case dynamics drawn from the set of possible transition probabilities. Additionally, Xu and Mannor, 2006 learns a policy that maximizes a combination of a performance and robustness measure, where the robustness component captures the worst-case reward for the evaluated action. By optimizing against the worst-case outcomes in the reward or transition dynamics, these approaches produce policies that are robust but often conservative, sometimes resulting in reduced performance.

Many formulations that address transition and reward uncertainty are limited to tabular settings and do not easily extend to nonlinear function approximators or continuous state-action spaces. Another line of research in robust RL instead focuses on robustness to adversarial or noisy actions, which can be applied in these more complex settings. The work of Tessler et al., 2019 introduces the Noisy Action Robust MDP, where the standard policy is replaced with a joint policy π_{mix} that perturbs the intended action through an adversarial policy. The resulting optimization is formulated in a minimax framework for the mixed policy. Similarly, Tan et al., 2020 defines a robust optimization objective in which adversarial perturbations are applied to actions in order to minimize the expected reward before policy optimization occurs. Much like the transition and reward methods, these methods aim to learn policies that perform well under worst-case action perturbations.

The objective proposed in this work draws a significant amount of inspiration from robust RL techniques. However, rather than optimizing for the worst-case scenario like the methods described above, the approach is to optimize with an added term that quantifies the robustness of actions to perturbations. This formulation aims to produce policies that remain effective under environmental uncertainty without becoming overly conservative, thereby balancing robustness and performance.

2.3 Offline Reinforcement Learning

Offline reinforcement learning (Prudencio et al., 2023) focuses on learning a policy π entirely from a fixed dataset $\mathcal{D}$ that has been previously collected by an exploration or behavior policy π_β , without any additional interaction with the environment (see Figure 2.1). This approach contrasts with online RL, where the agent continually collects new experience while learning. Despite this distinction, the main objective of offline RL is the same, which is to learn a policy that maximizes expected cumulative reward. Because training is performed on static, batched data, offline RL is often more sample-efficient and can converge faster than online RL, where interactions with the environment may be costly and slow down the learning process.

However, a central challenge with offline RL is distribution shift. During training, the learned policy π may select actions or reach states that were rarely or never encountered in the dataset $\mathcal{D}$. This mismatch between the training data distribution and the states and actions encountered by the learned policy can result in unreliable value estimation, which in turn may lead to poor policy performance when deployed. To address this issue, prior work has developed a range of strategies. Techniques such as importance sampling (Precup et al., 2000) help correct the bias induced by off-policy evaluation, while regularization methods (Kumar et al., 2020) can encourage desirable behaviors in the learned policy that may not be sufficiently represented in the dataset $\mathcal{D}$. Similarly, policy constraints (Fujimoto et al., 2019) can be used to keep the learned policy π close to the dataset $\mathcal{D}$, improving the reliability of value estimation.

Although the method presented in this paper incorporates principles from offline RL, it does not implement a pure offline approach. The learned policy π is subsequently deployed in the environment to acquire more data, which is then used to iteratively refine the reward model and policy. Despite this, the strategy of regularization formed a primary inspiration for this work. The combination of offline training with online data collection gives this method the efficiency of offline RL while also addressing the limitations of static datasets.

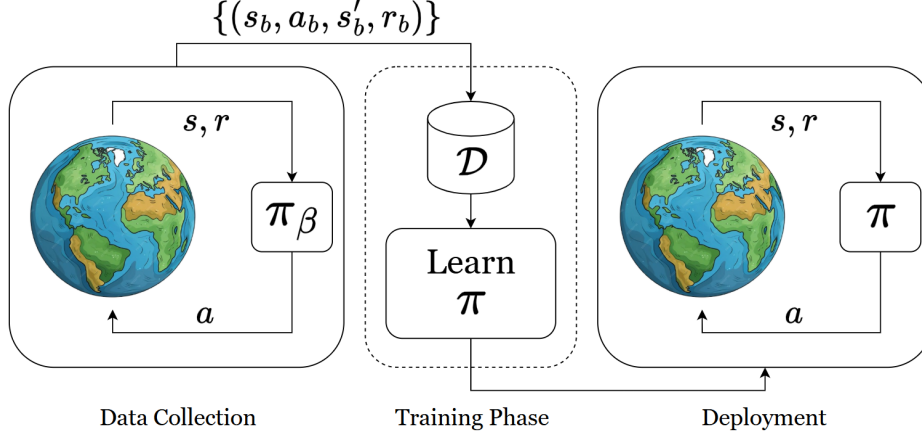


Figure 2.1: Offline Reinforcement Learning. An exploration policy π_β interacts with the environment to collect a dataset of state-action transitions and rewards, which is stored in the buffer $\mathcal{D}$. This dataset is used to train a policy π , which is then deployed in the environment without further interaction.

2.4 Model-Based Reinforcement Learning

Model-based reinforcement learning (Luo et al., 2024) is a class of methods that learn a model of the environment’s dynamics or transition function, which predicts the resulting state of actions taken by the agent. The learned transition model allows the agent to simulate future states and generate synthetic data without interacting with the real environment. This is particularly useful in domains where real world interactions are costly, dangerous, or time consuming. By leveraging a learned model, these methods significantly improve sample efficiency since the agent can perform planning and optimization on a combination of real and simulated experiences. A basic model-based RL pipeline is illustrated in Figure 2.2.

A key challenge in model-based RL is model accuracy and uncertainty. Several approaches address this issue through uncertainty-aware modeling, such as ensemble methods which train multiple independent models. One approach, proposed in Matsushima et al., 2020, trains an ensemble of K dynamics models and randomly selects one for each rollout, which helps reduce overfitting and increases robustness to individual errors. Additionally, Yu et al., 2020 learns N reward models and estimates the reward as the

ensemble mean penalized by the maximum standard deviation to incentivize the policy to avoid regions of high uncertainty.

In addition to ensemble methods, many model-based RL methods adopt iterative model refinement and Dyna-style (Sutton, 1991) techniques to continuously improve the dynamics model and policy. Works such as Feinberg et al., 2018 and Nagabandi et al., 2018 follow this paradigm by alternating between model learning and data collection with the optimized policy to progressively improve performance. This leads to more effective agents, especially in complex environments.

Although the proposed approach is related to model-based RL, it does not explicitly model a transition function. Instead, the method focuses on learning a reward model, which estimates the expected return of actions directly. Using the reward predictions, the policy can be optimized without the need for modeling state transitions. Despite this distinction, the approach adopts ideas from model-based RL, including the use of ensemble networks to capture uncertainty and iterative model improvement to enhance model accuracy.

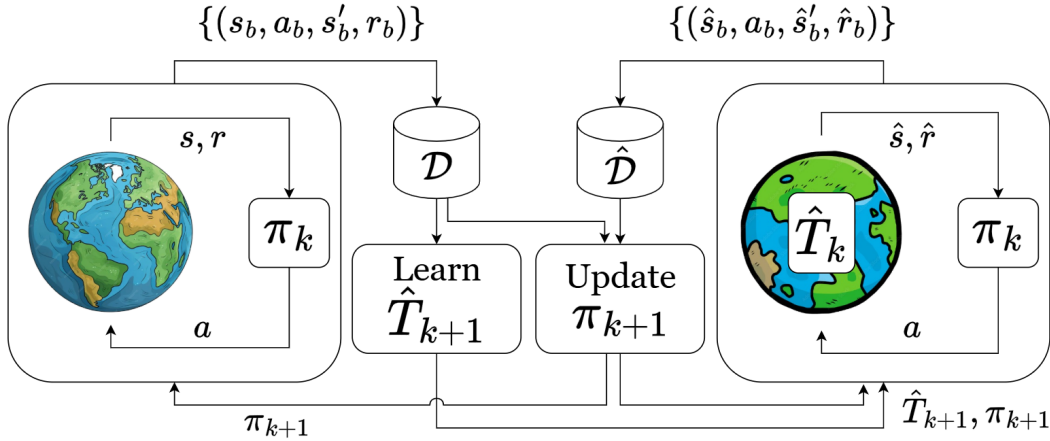


Figure 2.2: Model-Based Reinforcement Learning. A policy π_k is executed in the real world to collect data, which populates the real world buffer $\mathcal{D}$. This data is used to learn an approximate transition function $\hat{T}_k$. The learned transition function is used to generate a simulated dataset $\hat{\mathcal{D}}$, which is combined with the real-world dataset $\mathcal{D}$ to update the policy π_{k+1} . This iterative process improves sample efficiency by utilizing simulated data while grounding learning in real world experience.

CHAPTER 3

FOUNDATIONAL CONCEPTS

This chapter reviews the fundamental principles and methods that define this work, providing the necessary background for understanding the techniques presented later. Section 3.1 introduces contextual bandits, a framework for context-aware, action-selecting agents. Section 3.2 discusses policy gradient reinforcement learning, highlighting the basics of policy optimization with the gradient of the learning objective. Section 3.3 describes the process of modeling the expected reward function, emphasizing the estimation of unknown reward distributions through self-supervised learning. Finally, Section 3.4 introduces regularization in RL, explaining how it can improve the policies learned from static datasets. Together, these preliminaries establish the theoretical and practical foundation for the approach developed in this work.

3.1 Contextual Bandits

The multi-armed bandit (Slivkins, 2024) framework is one of the most widely used models for decision making under uncertainty. In this setting, a single agent chooses from K arms or actions in each round and receives a reward from an unknown distribution associated with the chosen arm. After observing the reward, the agent repeats this process by selecting from the same set of arms, making it a sequential

problem with inherent uncertainty. The objective is to both learn the reward distribution of each arm and to maximize the expected reward.

A common extension of the multi-armed bandit model is the contextual bandit framework (Langford & Zhang, 2007). Similar to the previous formulation, the agent chooses from a set of K arms or actions; however, in this case, the reward distribution now depends on an observed context. Despite this added structure, the goal of the contextual bandit problem remains the same as the multi-armed bandit, to maximize reward in a stochastic and unknown environment. The contextual bandit (CB) can be formally defined with the tuple

$$\text{CB} = \langle \mathcal{X}, \mathcal{A}, R \rangle, \quad (3.1)$$

where $\mathcal{X}$ denotes the context space, $\mathcal{A}$ represents the action space, and R is the reward function. These components are explained below:

- **Context Space:** The context space $\mathcal{X}$ is the set of all possible contexts in the environment. In each round, a context $x \in \mathcal{X}$ is observed, representing the current state of the environment. The context x is drawn from an underlying context distribution ρ_x . The context may be discrete, continuous, or multi-dimensional, and is assumed to be fully observable.
- **Action Space:** The action space $\mathcal{A}$ is the set of all possible actions available to the agent. Given a context x , the agent selects an action $a \in \mathcal{A}$ and executes it in the environment. The action may also be discrete, continuous, or multi-dimensional.
- **Reward Function:** The reward function R is used to determine the reward r after each round. The reward function R is a function of the context x and action a and returns a distribution of real-valued rewards $R : \mathcal{X} \times \mathcal{A} \rightarrow \Delta_{\mathbb{R}}$. The observed reward r is then sampled from the reward function $r \sim R(\cdot|x, a)$ during each round. The underlying reward distribution induces an expected reward function $\mathcal{R}(x, a) = \mathbb{E}_{r \sim R(\cdot|x, a)}[r]$, which represents the average reward for each context-action pair.

In order to determine which actions to take in each context, the agent implements a policy π , which defines a mapping from the context space to the action space. A policy can either be stochastic or deterministic, depending on the desired behavior. A stochastic policy maps a given context x to a probability distribution over the actions $\pi : \mathcal{X} \rightarrow \Delta_{\mathcal{A}}$. To select an action, the agent samples the action a from the resulting distribution $a \sim \pi(\cdot|x)$. In the deterministic case, the policy maps each context to a single action $\pi : \mathcal{X} \rightarrow \mathcal{A}$, such that $a = \pi(x)$ with a probability of 1.

3.2 Policy Gradient Reinforcement Learning

Reinforcement learning (Sutton, Barto, et al., 1998) is one of the most extensively studied paradigms in artificial intelligence. It can be described as the process of an agent learning rational, favorable behavior through interactions with its environment. A wide variety of techniques fall under the umbrella of RL, but they are generally categorized into two broad classes: model-based and model-free methods. In model-based RL (see Section 2.4), the agent learns a model of the environment and uses this model for planning or predicting outcomes. In contrast, model-free RL bypasses the need for a model and aims to learn through direct interaction with the environment. Model-free methods are highly popular due to their simplicity, robustness, and ability to scale to complex domains.

Furthermore, model-free RL methods can be divided even further into policy gradient, value-based, and actor-critic techniques. In policy gradient approaches, the agent directly optimizes its policy by interacting with the environment and using the observed rewards to guide policy updates. These approaches are well-suited for continuous action spaces, as they focus solely on the policy, but are generally restricted to being on-policy due to the nature of policy optimization. In contrast, value-based RL methods focus on learning a function that estimates the value of states or actions, which is then used to guide action selection. These methods are typically more sample-efficient than policy gradient techniques because they can reuse past data, but they can struggle with large or continuous state and action spaces. Finally, actor-critic methods combine the strengths of policy gradient and value-based approaches. In this framework, the

actor represents the policy and the critic refers to the value function. The feedback of the critic is used to update the actor, leading to more stable and efficient learning compared to either method alone.

Despite the wide variety of RL algorithms, the underlying learning objective remains the same: to find a policy π that maximizes the expected reward. The most general form of the learning objective is defined with the Markov decision process (MDP) framework (Bellman, 1957), and is commonly expressed as

$$J(\pi) = \mathbb{E}_{s_0 \sim \rho_s, a_t \sim \pi(\cdot|s_t), s_{t+1} \sim \mathcal{T}(\cdot|s_t, a_t)} \left[\sum_{t=0}^{\infty} R(s_t, a_t) \right], \quad (3.2)$$

where the initial state s_0 is sampled from the state distribution ρ_s , actions a_t are sampled from the stochastic policy π , resulting states s_{t+1} are sampled from the stochastic transition function $\mathcal{T}$, and rewards are given by the reward function R which is assumed to be deterministic.

To adapt this learning objective to the contextual bandit setting, some changes need to be made. Since contextual bandits involve no state/context transitions and no notion of sequential decision-making, the transition function $\mathcal{T}$ and the summation over all time steps in an episode can be removed. Additionally, the reward function is typically modeled as stochastic rather than deterministic, with observed rewards sampled as $r \sim R(\cdot|x, a)$. This can be handled either by taking an expectation over sampled rewards or by directly using the expected reward function $\mathcal{R}$. The latter simplifies the formulation while remaining consistent with the original deterministic reward assumption, allowing scalar reward values to be used directly.

In addition to the following changes, let's consider the implementation of a deterministic policy, where each context x is mapped to a fixed action $a = \pi(x)$. Under these modifications, the learning objective becomes

$$J(\pi) = \mathbb{E}_{x \sim \rho_x} \left[\mathcal{R}(x, \pi(x)) \right], \quad (3.3)$$

where contexts x are sampled from the context distribution ρ_x , rewards are given by the expected reward function $\mathcal{R}$, and actions are deterministically selected by the policy π . Although contextual bandits may employ either stochastic or deterministic policies, this work focuses on deterministic policies; therefore,

Equation 3.3 is adopted as the learning objective. Under this formulation, the optimal policy π^* is defined as the policy that maximizes expected reward:

$$\pi^* = \arg \max_{\pi} J(\pi). \quad (3.4)$$

With the advancement of deep reinforcement learning (Y. Li, 2017), it has become increasingly common to represent policies using neural networks. A neural network (Gurney, 2018) can be described as a parameterized function composed of multiple layers and interconnected nodes, where each layer applies weights, biases, and activation functions. After receiving an input, the network propagates it through its layers to produce an output. In the RL setting, the neural network acts as the policy, taking a state or context representation as input and producing an action or distribution over actions as output. The use of these networks enables highly expressive, nonlinear function approximation, allowing policies and value functions to handle high-dimensional and complex domains. When a policy is parameterized by a neural network with parameters ϕ , it is denoted as π_ϕ . The learning objective can then be written as a function of these parameters:

$$J(\phi) = \mathbb{E}_{x \sim \rho_x} [\mathcal{R}(x, \pi_\phi(x))]. \quad (3.5)$$

Importantly, the objective itself has not changed, but instead, it is now expressed in terms of the parameters ϕ that represent the policy. This shift is crucial, as it transforms the reinforcement learning problem into an optimization problem over the parameter space.

This change gives rise to the central idea behind policy gradient methods, which is to optimize the parameters ϕ using the gradient of the objective. More specifically, the goal is to iteratively update the parameters ϕ in the direction of the gradient $\nabla_\phi J(\phi)$, which quantifies how changes in the parameters affect the expected reward. For the contextual bandit setting, the deterministic policy gradient can be expressed as a modification of the original formulation in Silver et al., 2014:

$$\nabla_\phi J(\phi) = \mathbb{E}_{x \sim \rho_x} [\nabla_\phi \pi_\phi(x) \cdot \nabla_a \mathcal{R}(x, a) \big|_{a=\pi_\phi(x)}]. \quad (3.6)$$

Like Equation 3.3, the deterministic policy gradient has been adapted from the MDP framework. In the contextual bandit formulation above, the immediate expected reward function $\mathcal{R}(x, a)$ has replaced the Q-function $Q(s, a)$. The Q-function returns the total discounted sum of future rewards starting from state s and taking action a . The Q-function captures transition dynamics and is required to propagate future rewards back to earlier actions. However, contextual bandits have no state or context transitions, so the policy does not need to account for future outcomes.

Policy gradient methods use the gradient of the learning objective to directly optimize the policy parameters. More specifically, the algorithm performs gradient ascent, adjusting the policy parameters in the direction that increases the expected reward. At each update step, the parameters are adjusted according to

$$\phi_{k+1} = \phi_k + \lambda \cdot \nabla_{\phi_k} J(\phi_k), \quad (3.7)$$

where λ is the learning rate. This update effectively pushes the policy toward actions that yield higher rewards, reinforcing behaviors that are more likely to succeed.

In practice, the gradient of the learning objective cannot be calculated exactly because it depends on the full distribution of contexts and rewards, which is generally unknown. Instead, it must be estimated from sampled interactions with the environment. This can be done by executing the current policy π_ϕ in the real world and adding the observed context, action, and reward tuples into a dataset $\mathcal{D}_{\pi_\phi} = \{(x_b, a_b, r_b)\}_{b \in \mathcal{D}_{\pi_\phi}}$. This dataset provides a finite sample of the policy's behavior and its effects on the environment, which can be used to estimate the gradient. The deterministic policy gradient can then be approximated as

$$\nabla_\phi J(\phi) \approx \frac{1}{\|\mathcal{D}_{\pi_\phi}\|} \sum_{b \in \mathcal{D}_{\pi_\phi}} \nabla_\phi \pi_\phi(x_b) \cdot \nabla_a \mathcal{R}(x_b, a_b) \Big|_{a_b = \pi_\phi(x_b)}. \quad (3.8)$$

This expression is an unbiased Monte Carlo estimate of the true gradient. Although this estimate does not yield the exact gradient, it is sufficiently accurate to guide parameter updates in policy gradient methods, allowing the agent to iteratively improve its policy.

3.3 Reward Modeling

The reward function R for a contextual bandit, defined in Section 3.1, is stochastic and returns a distribution over real-valued rewards. After taking action a in context x , the observed reward r is sampled from this distribution $r \sim R(\cdot|x, a)$. The expected reward function $\mathcal{R}$ can be used equivalently to represent the sampled rewards, but it instead returns a scalar value. Because of this, if the expected reward function is known, the policy can use it directly for optimization by calculating the gradient with Equations 3.6 or 3.8.

In many practical settings, however, the true reward function is unknown and must be inferred through interactions with the environment. One approach is to learn a policy directly from past experience and utilize importance sampling (Precup et al., 2000). Since this experience is collected under an exploration policy π_β , whose action probabilities differ from the policy π_k being optimized, the expected reward cannot be accurately estimated without adjustment. Importance sampling addresses this by weighting observed actions according to how likely the trained policy π_k would select them, thereby producing a more accurate estimate of the expected reward under the target policy π_k .

Although importance sampling is powerful, it struggles with deterministic policies and continuous action spaces. An alternative approach is to use the collected data to learn an approximate expected reward function $\hat{\mathcal{R}}$, and use this in place of the true expected reward function $\mathcal{R}$ for policy optimization. Under the exploration policy π_β , the agent observes tuples of the form (x_b, a_b, r_b) where x_b is the context, $a_b \sim \pi_\beta(\cdot|x_b)$ is the action taken, and r_b is the observed reward. These interactions can be used to generate a dataset $\mathcal{D} = \{(x_b, a_b, r_b)\}_{b \in \mathcal{D}}$. Modeling the approximate reward function then becomes a supervised learning problem.

In supervised learning (Cunningham et al., 2008), the goal is to obtain a model that predicts the true labels of all input-output pairs in a dataset. To measure the discrepancy between the true labels y corresponding to input x and predicted labels $\hat{y}$ from the approximate model $\hat{f}(x)$, a loss function $\mathcal{L}(y, \hat{f}(x))$ is introduced (C. Li et al., 2025). The empirical risk $\mathcal{L}(\mathcal{D}, \hat{f})$ or the average loss of the model

over an entire dataset is then defined as

$$\mathcal{L}(\mathcal{D}, \hat{f}) = \frac{1}{\|\mathcal{D}\|} \sum_{b \in \mathcal{D}} \mathcal{L}(y_b, \hat{f}(x_b)). \quad (3.9)$$

This quantity measures the performance of the model on the observed data. The model that best fits the data $\hat{f}^*$ is obtained by minimizing the empirical risk:

$$\hat{f}^* = \arg \min_{\hat{f}} \mathcal{L}(\mathcal{D}, \hat{f}). \quad (3.10)$$

In practice, it is very common to represent the model using a neural network. For a model $\hat{f}_\theta$ parameterized by θ , the loss function can be rewritten as $\mathcal{L}(y, \theta)$, so that it is now a function of the true labels y and the parameters θ that represent the approximate model. With this change, Equation 3.10 is reframed as finding the parameters that minimize the empirical risk. This optimization problem is typically solved with gradient descent, where the parameters are iteratively updated in the direction of the negative gradient of the empirical risk. The parameters θ are updated according to the following rule:

$$\theta_{k+1} = \theta_k - \lambda \cdot \nabla_{\theta} \mathcal{L}(\mathcal{D}, \theta_k), \quad (3.11)$$

where λ is the learning rate.

In the setting of learning a model to approximate the expected reward function, the goal is to predict the true expected reward for each context-action pair. This means the reward model is trained by minimizing the loss between the true observed rewards and predicted expected rewards: $\mathcal{L}(r_b, \hat{\mathcal{R}}_\theta(x_b, a_b))$. Because rewards are obtained directly from interactions with the environment, no manual annotation is required. In this case, reward modeling can be viewed as a form of self-supervised learning (Jaiswal et al., 2020), an effective branch of supervised learning that avoids the cost of labeling large datasets. For a self-annotated dataset $\mathcal{D}$, the optimal parameters θ of the approximate expected reward function $\hat{\mathcal{R}}$ are

obtained by iteratively minimizing the empirical risk over a sampled mini-batch B :

$$\theta^* = \arg \min_{\theta} \frac{1}{\|B\|} \sum_{b \in B} \mathcal{L}(r_b, \hat{\mathcal{R}}_{\theta}(x_b, a_b)). \quad (3.12)$$

Once trained, this model can be used for policy optimization, enabling sample-efficient learning without access to the true expected reward function.

When applying supervised learning, it is important to choose a loss function that is appropriate for the given problem. In the context of learning an approximate expected reward function $\hat{\mathcal{R}}$, the model is trying to capture the true underlying stochastic reward function R , which returns a probability distribution over reward outcomes. Because of this, cross-entropy (CE) loss (Mao et al., 2023) is suitable, as it is widely used for probability models and classification tasks. The most general form of CE loss is categorical cross-entropy loss $\mathcal{L}_{CCE}$, which is defined as

$$\mathcal{L}_{CCE}(\theta) = - \sum_{c \in C} y_c \cdot \log(\hat{y}_c), \quad (3.13)$$

where C is the set of categories for the labels. The equation above is for a single-instance loss, so the corresponding empirical risk $\mathcal{L}_{CCE}$ for a mini-batch B is

$$\mathcal{L}_{CCE}(\theta) = - \frac{1}{\|B\|} \sum_{b \in B} \sum_{c \in C} y_{b,c} \cdot \log(\hat{y}_{b,c}). \quad (3.14)$$

Minimizing this objective encourages the reward model to assign high probability to observed rewards and low probability to others, effectively learning a distribution over rewards.

In many practical settings, the reward is binary, taking one of two labels (success or failure). In this case, binary cross-entropy (BCE) loss is appropriate. The per-sample BCE loss $\mathcal{L}_{BCE}$ is

$$\mathcal{L}_{BCE}(\theta) = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})], \quad (3.15)$$

and the corresponding empirical risk $\mathcal{L}_{BCE}$ over a mini-batch B is

$$\mathcal{L}_{BCE}(\theta) = -\frac{1}{\|B\|} \sum_{b \in B} y_b \cdot \log(\hat{y}_b) + (1 - y_b) \cdot \log(1 - \hat{y}_b). \quad (3.16)$$

After training a reward model with BCE loss, the output represents the probability of receiving each of the two possible reward outcomes. For a binary reward system with values of 0 or 1, the expected reward simplifies to the probability of receiving 1:

$$\mathbb{E}_{r \sim R(\cdot|x,a)}[r] = 0 \cdot Pr(0|x, a) + 1 \cdot Pr(1|x, a) = Pr(1|x, a)$$

Since a reward of 1 is generally associated with a successful action, the reward model can be interpreted as outputting the probability of success.

3.4 Regularization in Offline Reinforcement Learning

Although offline reinforcement learning is effective, it is not without limitations. A key challenge arises from the fact that optimization is performed on a static dataset. This can lead to overfitting, where the learned policy performs well on the observed data but fails to generalize to unseen situations when deployed in the real world. These failures arise because the policy exploits biases in the dataset, which may be incomplete or does not sufficiently cover the relevant state and action spaces. This is particularly problematic because offline RL does not have the ability to correct these errors with environmental feedback like standard online RL algorithms.

To address these limitations, a common approach is to incorporate regularization into the learning process. In offline RL, regularization refers to the addition of an extra term or function to the standard RL objective. This term is designed to penalize or encourage specific states, actions, or characteristics, effectively shaping the learned policy towards behaviors that are not fully represented in the dataset. The main idea is that the dataset is limited or biased, and the regularization term serves as a corrective mecha-

nism, guiding the policy towards more favorable outcomes. Formally, for a policy π_ϕ parameterized by ϕ and a regularization function $\mathcal{R}(\cdot)$, the regularized objective can then be written as

$$J(\phi) = \mathbb{E}_{x \sim p_x(\cdot)} \left[\mathcal{R}(x, \pi_\phi(x)) \right] + \mathcal{R}(\cdot), \quad (3.17)$$

where the regularization term $\mathcal{R}(\cdot)$ can depend on any relevant variables, including the context x , action a , or even the policy π_ϕ itself.

The regularization function $\mathcal{R}(\cdot)$ is highly versatile and can be applied in multiple ways. For instance, adding a positive entropy term to the objective promotes stochasticity and encourages exploration in the policy, which is exemplified in Haarnoja et al., 2018. Alternatively, regularization can be used to constrain the learned policy to remain close to the dataset distribution. For example, Nachum et al., 2019 utilizes a negative f -divergence term, which penalizes deviations from the dataset and ensures the policy remains supported by the data. Overall, regularization is a powerful tool that can induce desirable behaviors and compensate for the limitations of offline RL, all while maintaining computational and sample efficiency.

CHAPTER 4

METHODOLOGY

This chapter presents the proposed approach for learning a policy using an approximate expected reward function. Section 4.1 describes how the reward model is learned, emphasizing strategies to improve accuracy and reliability of the expected reward function approximation. Section 4.2 then introduces an enhanced policy optimization framework, detailing ways to improve performance of the learned policy. These components form an effective approach for leveraging limited reward signals efficiently while maintaining stability and performance in uncertain and costly environments.

4.1 Approximating an Accurate Reward Model

If the true expected reward function $\mathcal{R}$ for a contextual bandit is unknown, a learned approximation of the expected reward function $\hat{\mathcal{R}}$ can be used in its place to directly apply policy gradient methods. As detailed in Section 3.3, learning a reward model consists of utilizing a dataset $\mathcal{D}$ defined as $\mathcal{D} = \{(x_b, a_b, r_b)\}_{b \in \mathcal{D}}$. The reward serves as the output label, and self-supervised learning is performed on the context-action inputs to predict the corresponding expected reward.

Although this basic process can produce effective results, additional strategies are employed to further improve the accuracy and robustness of the approximate reward model. First, an effective data collection strategy is used to construct a dataset with broad coverage of the context-action space and a diverse range

of reward values to guide model learning. Second, the approximate expected reward function is modeled with an ensemble of networks, leading to improved predictive performance and introducing a mechanism to quantify and penalize uncertainty. Together, these components enable the learned expected reward function to produce more accurate and reliable reward estimates for context-action pairs. This accuracy is crucial, as the approximate expected reward model is subsequently used to guide and optimize the policy.

4.1.1 Initial Data Collection Strategy

To produce an accurate model with supervised learning, it is typically crucial to train with large and representative datasets, as it enables the model to capture the relationship between inputs and outputs across a wide range of scenarios. However, in many real world domains, producing large-scale datasets is often time-consuming and expensive, especially when manual annotation is required. Because of this, when working with limited data, it is essential to maximize the diversity of the training dataset (Gong et al., 2019). This diversity improves coverage of the input space and allows the model to generalize effectively without access to huge quantities of data.

Reinforcement learning is commonly applied to robotics despite the high cost of data collection, as it provides a natural framework for learning from interaction. In practice, many of these systems allow for human intervention through teleoperation or external control, which can be used to produce expert demonstrations that yield high reward behaviors. This gives rise to learning methods like imitation learning (Hussein et al., 2017), or more specifically kinesthetic teaching (Billard et al., 2006), where policies are trained to replicate the demonstrated actions. While these methods can produce effective policies, their goal is to mimic the demonstrated behavior rather than to infer and optimize an underlying reward function. As a result, they are limited to the expert data distribution and may fail to generalize or improve upon it. Nevertheless, human-generated data provides a valuable source of information that can be leveraged beyond pure imitation.

To construct a dataset that is sufficiently diverse, the proposed approach combines data generated by a human expert policy π_E and a random exploration policy π_β to form an initial dataset $\mathcal{D}_0$. The

term “expert” is used loosely here; while these actions should produce high rewards, they do not need to be strictly optimal. These samples provide a positive learning signal to identify desirable regions of the reward space for policy optimization. In contrast, the exploration policy π_β is likely to produce lower-reward actions, but its primary purpose is to ensure broad coverage of the context and action space. This exploratory data exposes the model to a wider range of outcomes, including failure cases, which improves overall data diversity. By combining expert and exploratory data, the resulting dataset achieves both reward diversity and context-action coverage. This balance is important for learning an approximate expected reward function that can be reliably used for downstream policy optimization.

4.1.2 Leveraging Ensemble Networks

When a dataset lacks sufficient diversity to adequately cover the input space, a learned model cannot be trusted in under-represented regions. While a narrowly distributed dataset will yield strong performance on in-distribution samples, this becomes problematic when the model is utilized for out-of-distribution samples. Specifically, if a model for the expected reward function is learned and used to guide policy optimization, the policy may explore these poorly represented regions, resulting in unreliable reward estimates and degraded performance.

A common strategy in supervised learning to mitigate this issue is to train an ensemble of models rather than relying on a single predictor. Averaging predictions across the ensemble often yields smoother and more accurate estimates than any individual model. Additionally, disagreement among the models provides a useful measure of uncertainty. This uncertainty can be incorporated in many ways, such as using the most pessimistic prediction within the ensemble or penalizing predictions according to the variance across models. Either way, ensemble methods can significantly improve robustness in the model.

In this work, an ensemble of N independently parameterized expected reward functions $\bar{\mathcal{R}}_{\theta_1}, \dots, \bar{\mathcal{R}}_{\theta_N}$ are trained. Each model is initialized with different parameters θ_i and trained on independently sampled mini-batches, which encourages diversity among the learned networks. The ensemble is used to compute

the mean of the expected reward μ_θ and the standard deviation σ_θ , defined as

$$\mu_\theta(x, a) = \frac{1}{N} \sum_{i=1}^N \bar{\mathcal{R}}_{\theta_i}(x, a), \quad (4.1)$$

$$\sigma_\theta(x, a) = \sqrt{\frac{1}{N} \sum_{i=1}^N (\bar{\mathcal{R}}_{\theta_i}(x, a) - \mu_\theta(x, a))^2}. \quad (4.2)$$

The mean μ_θ serves as the primary estimate of the expected reward, while σ_θ captures the level of disagreement across the ensemble. High variance indicates that the input lies in a region that is poorly represented in the dataset, which means the current prediction is less reliable. To account for this uncertainty, the uncertainty-penalized reward $\hat{\mathcal{R}}_\theta$ is defined as

$$\hat{\mathcal{R}}_\theta(x, a) = \mu_\theta(x, a) - \beta \cdot \sigma_\theta(x, a), \quad (4.3)$$

where β is the uncertainty coefficient, a parameter that controls the strength of the penalty. When used for policy optimization, larger values of β will lead to more conservative approximations of the expected reward, discouraging the policy from selecting actions in regions where the reward model is uncertain. The following (proof in Appendix A) justifies this reward model by showing that it does not overestimate the true rewards with high probability:

Theorem 1 (Probabilistic Bound of Reward Model). *Suppose the ensemble networks estimation method has a true mean $\mathcal{R}(x, a) = \mathbb{E}[\bar{\mathcal{R}}_{\theta_i}(x, a)]$ and common variance $\sigma^2 = \text{Var}(\bar{\mathcal{R}}_{\theta_i}), \forall i$. Then, the uncertainty-penalized reward model $\hat{\mathcal{R}}_\theta(x, a) = \mu_\theta(x, a) - \beta \sigma_\theta(x, a)$ satisfies $\hat{\mathcal{R}}_\theta(x, a) \leq \mathcal{R}(x, a)$ with probability $\geq 1 - \delta$ for any $\delta \in (0, 1)$ and $\beta \geq \sqrt{\frac{1}{N\delta}}$.*

This ensures that policy optimization is conservative in regions of the context-action space where the reward model is unreliable. The training procedure for the ensemble is outlined in Algorithm 1.

Algorithm 1 Train Expected Reward Ensemble

Inputs: Dataset $\mathcal{D}$, Number of expected reward networks N , Uncertainty coefficient β **Output:** Expected reward model $\hat{\mathcal{R}}_\theta$

- 1: Initialize N expected reward networks $\bar{\mathcal{R}}_{\theta_1}, \dots, \bar{\mathcal{R}}_{\theta_N}$ with random parameters $\theta_1, \dots, \theta_N$
 - 2: **for** $t = 1$ to T **do**
 - 3: **for** $n = 1$ to N **do**
 - 4: Sample an independent mini-batch: $B_n \subset \mathcal{D}$
 - 5: Update parameters with empirical risk: $\mathcal{L} = \frac{1}{\|B_n\|} \sum_{b \in B_n} \mathcal{L}(r_b, \bar{\mathcal{R}}_{\theta_n}(x_b, a_b))$
 - 6: **end for**
 - 7: **end for**
 - 8: Calculate uncertainty-penalized reward model: $\hat{\mathcal{R}}_\theta = \mu_\theta - \beta \cdot \sigma_\theta$
 - 9: **return** $\hat{\mathcal{R}}_\theta$
-

4.2 Improved Policy Optimization

After learning an approximate expected reward function, the next step is to learn a policy, which is achieved by policy gradient optimization (See Section 3.2). While standard policy gradient methods have demonstrated strong performance, they often rely on the assumption that the reward signal is accurate and that maximizing expected reward is sufficient to produce effective behavior.

Because the reward model is approximated, additional techniques are introduced to improve the reliability and robustness of the optimization process. First, a robustness-aware objective is proposed, which encourages the policy to select high reward and perturbation-resilient actions. Second, an iterative learning procedure is adopted, in which a policy trained offline is repeatedly deployed to collect new data, enabling continuous refinement of the reward model and policy. Collectively, these strategies lead to a more stable and reliable optimization process, resulting in improved policy performance. This is particularly important when relying on an approximate expected reward function trained from limited or imperfect data.

4.2.1 Perturbed Action Robustness Objective

In reinforcement learning, the main objective is to maximize expected reward, as formalized in Equation 3.3. In many scenarios, this objective is sufficient to produce intelligent agents. However, in real-world settings, optimizing expected reward alone may lead to suboptimal behavior. This can be attributed to several factors, including poor exploration, incomplete data, or uncertainty in the environment.

To address these limitations, various techniques can be applied. One common approach is regularization (See Section 3.4), where an additional term is incorporated into the RL objective to arouse certain behaviors in the policy being optimized. For example, regularization can promote improved exploration or bias the policy toward regions supported by reliable data. Regardless of the desired traits, the resulting agent is often more successful than one trained with the standard learning objective alone.

Another line of work focuses on explicitly handling uncertainty through robust RL, which is explained in Section 2.2. These methods aim to account for uncertainty in different components of the RL problem, including the reward function and action execution. For instance, policies may be optimized with respect to the worst-case reward function or under adversarial perturbations to the actions. While such approaches produce robust agents, they can also lead to overly conservative policies due to their emphasis on the worst-case outcomes.

Both of these techniques were considered when addressing uncertainty in both the environment and the approximate expected reward function. To mitigate these issues, a new learning objective is introduced to promote robustness to action perturbations. First, a robustness measure, called the Perturbed Action Robustness (PAR) function, is defined as

$$\eta(x, a) \triangleq \frac{1}{Z} \cdot \int_{-\delta_{\max}}^{\delta_{\max}} \hat{\mathcal{R}}(x, a + \delta) d\delta, \quad (4.4)$$

where $\delta_{\max}$ is the largest possible action perturbation that can be observed in the environment and Z is a normalizing constant. The normalizer is commonly defined as

$$Z \triangleq \int_{-\delta_{\max}}^{\delta_{\max}} d\delta = 2 \cdot \delta_{\max}. \quad (4.5)$$

By normalizing η with $\frac{1}{Z}$, the output is set to the same scale as the expected reward function and it becomes the average reward in the integration region. Intuitively, it measures the effectiveness of nearby actions, assigning larger values to actions that lie in high reward regions. This robustness function is incorporated into the standard objective to obtain the PAR objective,

$$J(\phi) = \mathbb{E}_{x \sim \rho_x(\cdot)} \left[(1 - \alpha) \cdot \hat{\mathcal{R}}(x, \pi_\phi(x)) + \alpha \cdot \eta(x, \pi_\phi(x)) \right], \quad (4.6)$$

where α is the robustness coefficient, a parameter that determines the importance of the PAR function. This objective encourages the policy π_ϕ to select actions that not only achieve high expected reward, but those that are also stable under action perturbations. As a result, the learned policy π_ϕ is less likely to exploit narrow peaks in the reward landscape and is more likely to converge to broad, high reward regions. The standard RL objective is recovered in the limit as $\alpha \rightarrow 0$. The corresponding policy gradient is

$$\nabla_\phi J(\phi) = \mathbb{E}_{x \sim \rho_x(\cdot)} \left[\nabla_\phi \pi_\phi(x) \cdot \nabla_a \left((1 - \alpha) \cdot \hat{\mathcal{R}}(x, a) + \alpha \cdot \eta(x, a) \right) \Big|_{a=\pi_\phi(x)} \right]. \quad (4.7)$$

Much like the standard deterministic policy gradient shown in Equation 3.6, it is often intractable to calculate this expectation exactly. However, it can be estimated with Monte Carlo sampling, exemplified by the following

$$\nabla_\phi J(\phi) \approx \frac{1}{\|B\|} \sum_{b \in B} \nabla_\phi \pi_\phi(x_b) \cdot \nabla_a \left((1 - \alpha) \cdot \hat{\mathcal{R}}(x_b, a_b) + \alpha \cdot \eta(x_b, a_b) \right) \Big|_{a_b=\pi_\phi(x_b)}, \quad (4.8)$$

where B represents the sampled batch of contexts. This approximation provides an effective learning signal to guide policy optimization under the proposed objective. The following establishes the convergence property of the gradient update (proof in Appendix B):

Theorem 2 (Convergence of Policy Parameters). *Assume the approximate reward model $\hat{\mathcal{R}}(x, a)$ and policy $\pi_\phi(x)$ are both twice continuously differentiable w.r.t. action a and parameter ϕ respectively, i.e., $\hat{\mathcal{R}}$ and $\nabla_a \hat{\mathcal{R}}$ are L_{R_1} - and L_{R_2} -Lipschitz continuous respectively (w.r.t. a), and $\pi_\phi(x)$ and $\nabla_\phi \pi_\phi(x)$ are L_{π_1} - and L_{π_2} -Lipschitz continuous respectively (w.r.t. ϕ). Then the sequence of parameters $\{\phi_k\}$ generated by the gradient ascent update rule $\phi_{k+1} = \phi_k + \lambda \nabla J(\phi_k)$ (Eq. 3.7 applied to the PAR objective in Eq. 4.6) converges to a stationary point (i.e., $\lim_{k \rightarrow \infty} \|\nabla J(\phi_k)\| = 0$) if the step size λ satisfies $0 < \lambda < \frac{2}{L}$, where $L = L_{\pi_2} L_{R_1} + L_{\pi_1} L_{R_2}$.*

4.2.2 Iterative Deployment of Trained Policy

One of the major problems with offline RL (Section 2.3) is its inability to improve its policy through interactions with the environment. Because learning is constrained to a fixed dataset, the learned policy is strongly dependent on the data collected by the exploration policy. When this data is incomplete or does not provide sufficient coverage of the context and action spaces, optimization may lead the policy to regions that appear optimal under the data distribution, but when deployed in the true environment, the policy underperforms.

Similarly, a major problem with model-based RL, which is detailed in Section 2.4, is the accuracy of the learned dynamics or reward model. Because these models are trained on previously collected experience, they inherit the same limitations as policies learned using offline RL. In regions where data is sparse or absent, model predictions become unreliable. This is particularly problematic when policy updates shift the agent towards these out-of-distribution states, where the reward estimates are likely inaccurate.

To tackle these coupled issues, the proposed method combines offline training with iterative online deployment for policy evaluation and model refinement. After optimization under the modified objective (Equation 4.6), the resulting policy π_{ϕ_i} is executed in the environment to generate a new dataset $\mathcal{D}_{\pi_{\phi_i}}$.

To improve exploration and to evaluate robustness, Gaussian noise is added to the continuous actions of the deterministic policy π_{ϕ_i} during deployment. The collected context-action-reward tuples are then aggregated with the existing dataset as follows:

$$\mathcal{D}_{i+1} = \mathcal{D}_i \cup \mathcal{D}_{\pi_{\phi_i}}, \quad (4.9)$$

resulting in a progressively expanding dataset that combines random exploration, expert data, and newly collected on-policy experience.

This aggregated dataset is subsequently used to retrain the expected reward model, allowing it to adapt to regions of the context-action space induced by the current policy. As a result, the expected reward approximator becomes progressively more accurate in high relevance regions. The improved expected reward model is then used to re-optimize the policy, either reinforcing the current actions or steering the policy away from them. This iterative refinement helps mitigate errors arising from both the policy and the learned model. The overall procedure is presented in Figure 4.1 and summarized in Algorithm 2. Theoretical analysis of the stability of this process can be found in Appendix C.

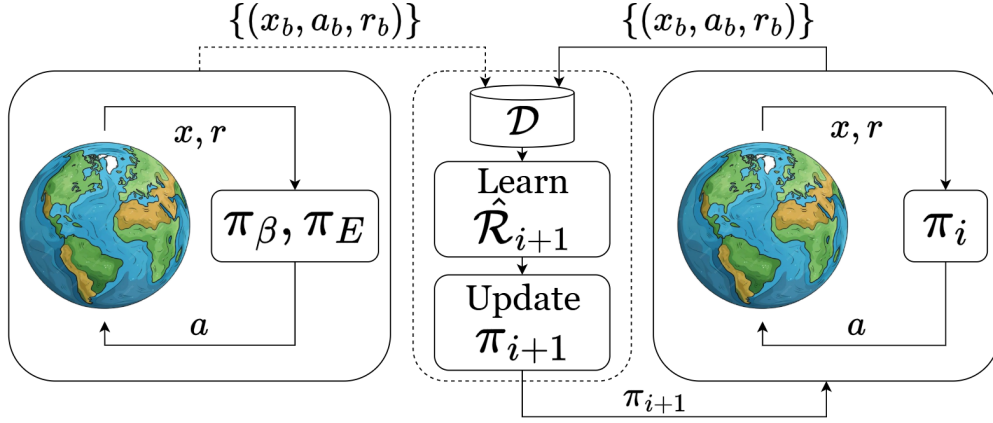


Figure 4.1: Proposed Policy Learning Framework. An exploration policy π_β and expert policy π_E interact with the environment to collect context-action-reward tuples, which are stored in the initial dataset $\mathcal{D}$. This dataset is used to train an approximate expected reward model $\hat{\mathcal{R}}_i$, which guides the optimization of a policy π_i . The learned policy π_i is deployed in the environment to generate on-policy data and adds it to the dataset $\mathcal{D}$. The reward model $\hat{\mathcal{R}}_{i+1}$ is updated with the augmented dataset, and a new policy π_{i+1} is optimized. This process repeats iteratively, progressively improving the reward model and policy.

Algorithm 2 Policy Optimization

Inputs: Number of reward networks N , Robustness coefficient α , Uncertainty coefficient β

Output: Policy π_ϕ

- 1: Initialize policy network π_ϕ with random parameters ϕ
 - 2: Fill initial dataset $\mathcal{D}_0$ with data from exploration policy π_β and from expert policy π_E
 - 3: **for** $i = 0$ to I **do**
 - 4: $\hat{\mathcal{R}}_\theta \leftarrow \text{TrainRewardModel}(\mathcal{D}_i, N, \beta)$
 - 5: **for** $t = 1$ to T **do**
 - 6: Sample mini-batch from current dataset: $B \subset \mathcal{D}_i$
 - 7: Update parameters with gradient:

$$\nabla_\phi J(\phi) = \frac{1}{\|B\|} \sum_{b \in B} \nabla_\phi \pi_\phi(x_b) \cdot \nabla_a ((1 - \alpha) \cdot \hat{\mathcal{R}}_\theta(x_b, a_b) + \alpha \cdot \eta(x_b, a_b))|_{a_b = \pi_\phi(x_b)}$$
 - 8: **end for**
 - 9: Execute π_{ϕ_i} and add interactions to $\mathcal{D}_{\pi_{\phi_i}}$
 - 10: Update dataset: $\mathcal{D}_{i+1} = \mathcal{D}_i \cup \mathcal{D}_{\pi_{\phi_i}}$
 - 11: **end for**
 - 12: **return** π_ϕ
-

CHAPTER 5

EXPERIMENTS

This chapter presents the experiments conducted on the proposed approach. Section 5.1 describes the simulation experimental setup in detail, including the environment and evaluation functions. Section 5.2 then details the physical experimental setup, outlining the robotic platform and task configuration. Section 5.3 introduces the robust RL baselines used for comparison, highlighting their relevance to the method. Finally, in Section 5.4, the proposed method is evaluated in both simulation and on the robotic manipulation task, with the results presented and analyzed. Together, these experiments provide an insightful assessment of the proposed approach, which shows improved performance over the robust RL baselines in the simulated and real-world settings.

5.1 Simulation Experimental Setup

To evaluate different components of the method in a simulated setting, the Buffalo Gym (Parker, 2025) environment was utilized. Buffalo Gym is a third-party gymnasium environment that allows experimentation on a variety of bandit formulations. The environment is single-state, but allows the user to generate stochastic reward functions, making it easy to perform analysis on different optimization procedures with a wide range of functions. Experiments were performed on the “Boundless Buffalo” which supports continuous action spaces, with some custom modifications to allow for predefined stochastic reward

functions. Five such reward functions were produced, shown in Figure 5.1 with the corresponding maximum action perturbations shown in Table 5.1. These functions were chosen specifically so that the reward landscape requires robustness for success. The fork of the Buffalo Gym repository can be found at the following link: <https://github.com/thinclub/buffalo-gym>.

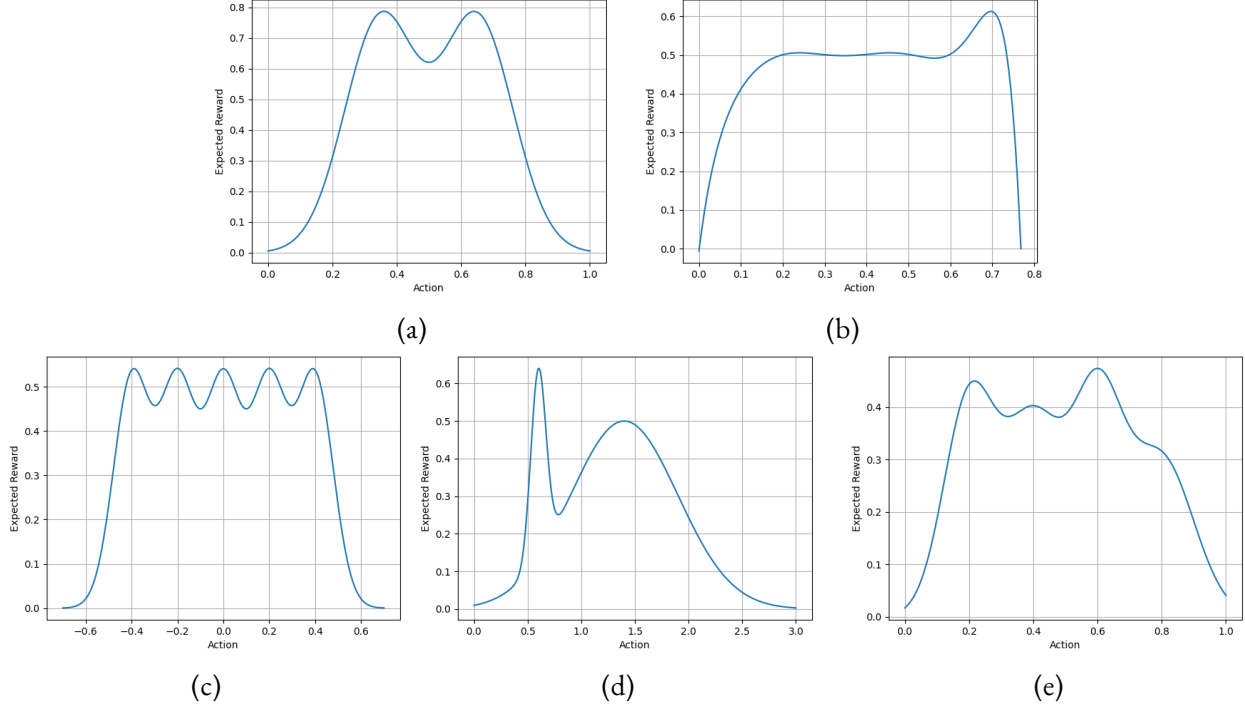


Figure 5.1: Buffalo Gym Expected Reward Functions. Each figure above represents one of the five expected reward functions used for evaluation.

Table 5.1: Maximum Action Perturbations. The maximum action perturbation $\delta_{\max}$ for each Buffalo Gym expected reward function is listed.

	Function 1	Function 2	Function 3	Function 4	Function 5
$\delta_{\max}$	0.25	0.19	0.35	0.75	0.25

5.2 Physical Experimental Setup

This section describes the experimental setup used to evaluate the proposed method in a robotic manipulation environment. The setup consists of camera-based perception system and a KUKA collaborative

robot equipped with an Ubiros gripper. The goal of the system is to autonomously sort recyclable objects within the workspace. The code for this project can be found at: <https://github.com/thinlab/recyclrl>.

5.2.1 Robotic Task Description

The proposed task is to autonomously sort recyclable items within a constrained workspace. The item set consists of aluminum cans and plastic bottles with varying degrees of deformation, which introduces variability in shape and grasp difficulty. The object manipulation is performed with a KUKA LBR iisy 3 r760 collaborative robotic arm equipped with an Ubiros Gentle Pro gripper at the end effector. The context of the workspace is captured using an overhead Oak-D S2 RGB-D camera, which provides synchronized color and depth images of the scene. These images are processed and encoded to form the input to the learned policy. The policy outputs continuous control actions for the manipulator, specifying the relative end effector placement. After executing the policy action with the corresponding motion trajectory, the robot automatically closes the gripper to secure the object, moves to the recycling bin, and opens the gripper to release any grasped object. An action is considered successful if the robot is able to securely grasp the assigned object and transport it to the recycling bin. The experimental setup is illustrated in Figure 5.2, the overall sorting procedure is summarized in Algorithm 3, and an example of robot a trajectory is shown in Figure 5.3.

Algorithm 3 Sorting

Input: Learned policy π_ϕ

```

1: while items in workspace do
2:   Camera captures RGB and Depth images of the workspace
3:   Images are processed to obtain context  $x$  and encoded context  $\hat{x}$ 
4:   Robot moves to "Home" position
5:   Determine action from policy:  $a \leftarrow \pi_\phi(\hat{x})$ 
6:   Robot executes action  $a$  by moving to relative end effector position
7:   Robot closes gripper to grab item
8:   Robot moves to "Bin" position
9:   Robot opens gripper to drop item
10: end while
```

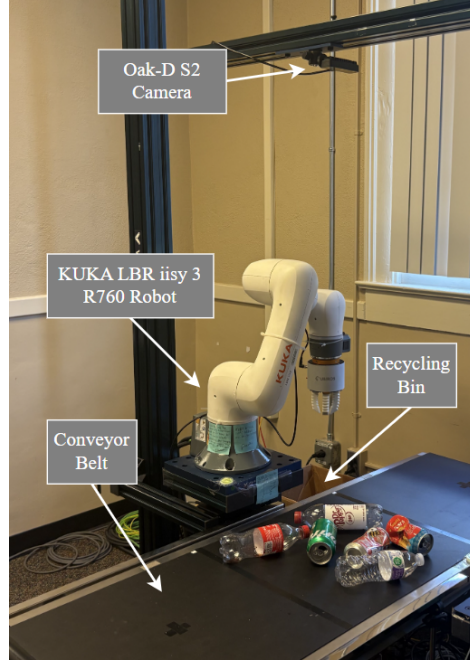


Figure 5.2: Physical Experimental Setup. A KUKA robotic manipulator is used to grab aluminum cans and plastic bottles located on a conveyor belt. An Oak-D RGB-D camera captures images of the scene, from which the 3D position and orientation of each item is calculated. With this information, the manipulator executes grasping actions to move the recyclable items to a recycling bin

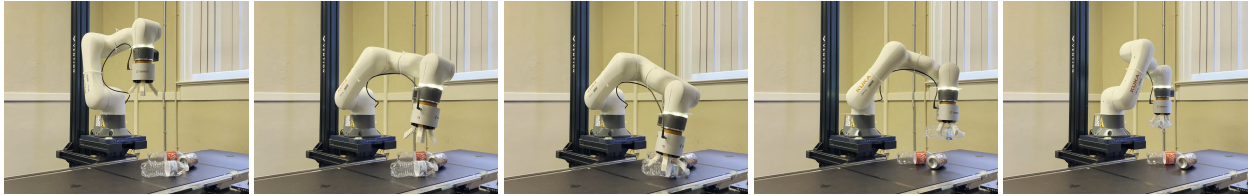


Figure 5.3: Robot Trajectory. An example trajectory is shown in the frames above. The robot starts at home, moves to the grasp position, grabs the bottle, and moves to the bin where it deposits the item.

5.2.2 Context Estimation

The context of the workspace is obtained using images from an overhead RGB-D camera. When the current context is requested, the camera captures an RGB image and a corresponding depth image of the workspace. The RGB image is passed through a YOLO v11 instance segmentation model, which produces pixel-wise masks of the aluminum cans and plastic bottles present in the scene. The YOLO model was trained on a combination of publicly available datasets containing annotated images of general cans and

bottles, as well as a custom dataset containing images collected directly from the workspace with manual annotation. Incorporating general and task-specific data enables the model to generalize and maintain accuracy for specific objects with the instance segmentation procedure.

After obtaining the segmented instances from the RGB image, principal component analysis (PCA) (Greenacre et al., 2022) is performed on the mask of each item. PCA provides an efficient method to calculate different geometric properties from the object masks, and is first utilized to compute the centroid. The centroid represents the average position of all the pixels in the object mask, effectively providing a 2D estimate of the object's center in the image plane. In addition to the centroid, PCA is utilized to calculate the principal axis of each mask, which is used to indicate the orientation of the each object in the image. Figure 5.4 provides a visual overview of this process, showing the initial RGB image, the extracted object masks, and the result of the PCA analysis on each mask.

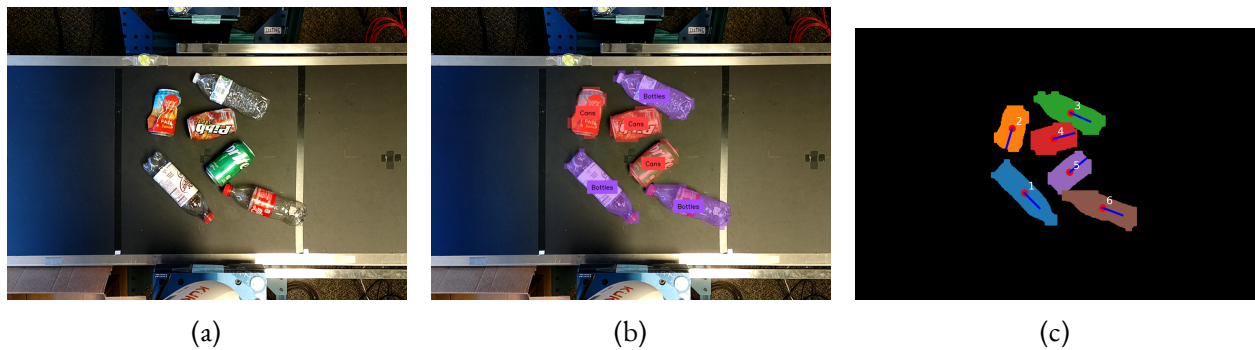


Figure 5.4: Context Estimation. Figure 5.4a shows the original RGB image. This image is passed through a YOLO v11 instance segmentation model to obtain masks of the bottles and cans, as shown in Figure 5.4b. Principal component analysis (PCA) is performed on the masks to determine the centroid and principal axes, shown in Figure 5.4c

The corresponding depth image is then used to recover the 3D position of the centroid from its 2D location in the image. Using the camera intrinsics and a standard pinhole model, the pixel corresponding to the centroid can be projected along a 3D ray. The depth value at the centroid's pixel represents the distance along this ray, allowing the computation of the object's centroid relative to the camera. This 3D position is transformed from the camera's local coordinate frame to the robot's global coordinate frame using ROS2 transformations.

The principal axis obtained from PCA defines the object’s yaw angle in the robot frame. However, for a complete 6D pose representation, the pitch angle must also be estimated. To compute the pitch, a two-point method is employed. Two points are selected near opposite ends of the object along the principal axis. The 3D positions of these points are recovered from the depth image using the same strategy applied to the centroid. Let the vertical difference between the two points be Δz , and let the planar displacement in the xy -plane be Δxy . The pitch angle is then computed by $\omega = \arctan \frac{\Delta z}{\Delta xy}$, which provides a measure of the object’s incline relative to the workspace plane. Due to assumed axial symmetry of the bottles and cans, the roll angle is assumed to be zero. Therefore, the full 6D pose of the each object is defined by the 3D position of the centroid and the estimated pitch and yaw angles.

5.2.3 Context Definition

Because the robot focuses on grasping one item at a time, the context representation only includes the 6D pose of the selected target. This approach greatly reduces the context space by concentrating on the object that is relevant for the current grasping action, rather than including the positions and orientations of all other objects, which is difficult when the number of items is dynamic. Therefore, the context captures the position and orientation of the active object in 3D space, along with basic contextual information about the scene. Formally the context is defined as:

$$\mathbf{x} = [x, y, z, \omega, \psi, t, n],$$

where x , y , and z represent the Euclidian coordinates of the object’s centroid, ω and ψ denote the pitch and yaw angles describing the object’s orientation, t represents the object type (e.g., can or bottle), and n denotes the number of objects currently present in the scene. Information about the object type t is necessary as the geometry of aluminum cans and plastic bottles differs. Due to the physical constraints of

the workspace, the following bounds are imposed on the context variables:

$$\begin{aligned} x &\in [0.20, 0.55], \quad y \in [-0.35, 0.35], \quad z \in [0.80, 0.95], \\ \omega &\in [-\frac{\pi}{2}, 0], \quad \psi \in [-\pi, \pi]. \end{aligned}$$

These bounds ensure that the context representation includes only feasible poses, preventing the robot from considering impossible positions or orientations during planning or learning.

Because the true context space is continuous, there are infinitely many possible contexts. To significantly reduce the size of the context space for the reward and policy networks, a discretized encoding is introduced. The workspace is first divided into two equal partitions along the x and y axes, producing four spatial quadrants. Each object is assigned a quadrant based on the position of its centroid, compressing all possible continuous x and y values into four discrete values. Then, the pitch angle ω range is divided into three uniform bins, and the yaw angle ψ range is divided into four uniform bins. Like the quadrant representation, the pitch and yaw bins compress the context representation into a coarse but still meaningful form. The object type t is preserved in the discretized context to maintain the distinction between different recyclable items, while the number of objects n is removed to simplify the encoding and because only a single target is considered. The resulting discretized state is compactly defined as

$$\hat{\mathbf{x}} = [q, \Omega, \Psi, t],$$

where $q \in \{1, 2, 3, 4\}$ denotes the spatial quadrant of the object, $\Omega \in \{1, 2, 3\}$ represents the discretized pitch bin, $\Psi \in \{1, 2, 3, 4\}$ denotes the discretized yaw bin, and t represents the object type. This encoding provides a simplified yet informative summary of the object and reduces the continuous context to a compact representation that is suitable for reward modeling and policy learning. Figure 5.5 shows the discretized workspace along with an example of an item mapped to the corresponding quadrant and orientation bins. Although an encoded, discretized context $\hat{\mathbf{x}}$ is produced, the true context $\mathbf{x}$ is preserved to build an absolute pose for the gripper from the relative action (see Section 5.2.4).

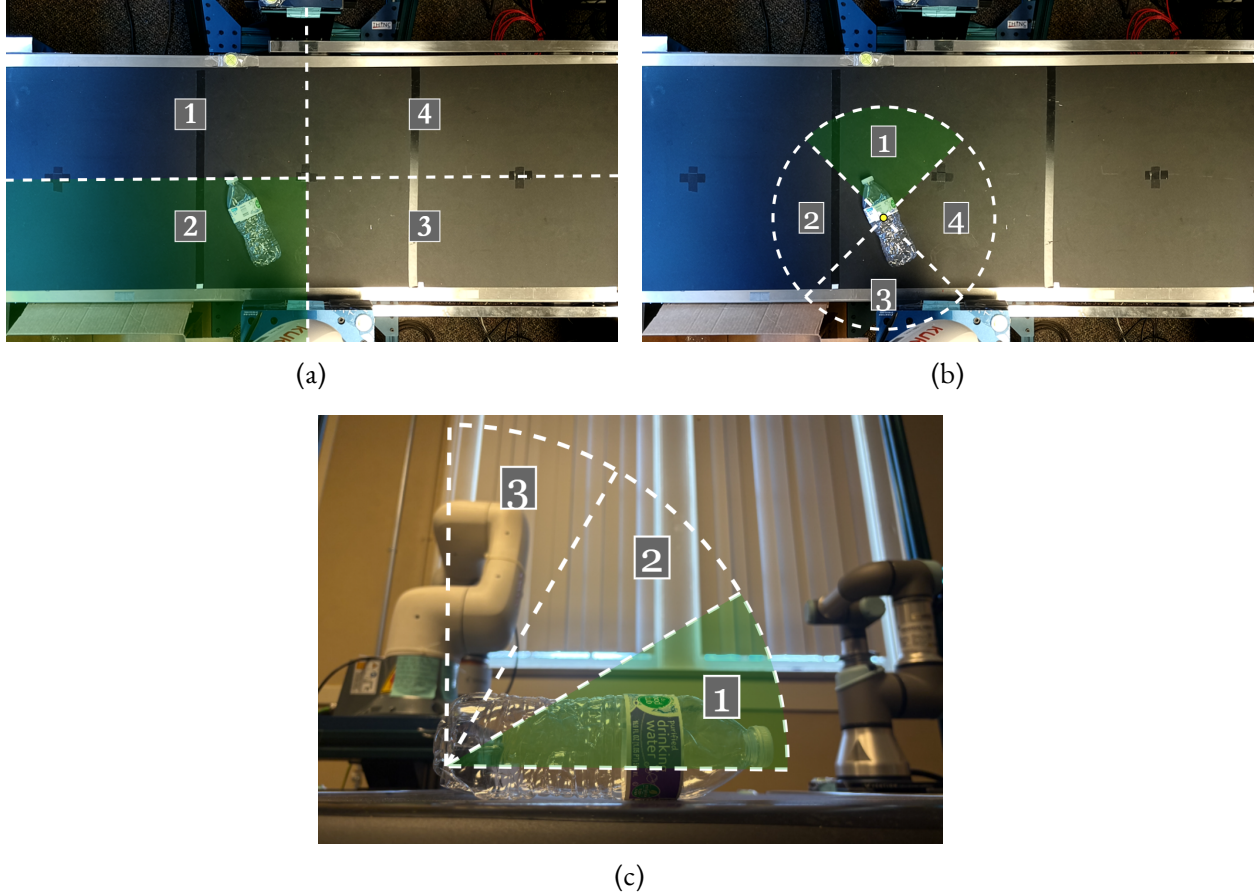


Figure 5.5: Discretized Workspace: Figure 5.5a shows the xy-plane of the workspace divided into four quadrants. In this example, the bottle is located in the second quadrant. Figure 5.5b illustrates the discretization of the possible yaw angles into four bins; the bottle falls within the first bin in this example. Similarly, the possible pitch angles are discretized into three bins, with the bottle located in the first bin, as shown in Figure 5.5c.

5.2.4 Action Definition

The actions for the sorting task define how the robotic manipulator moves prior to attempting a grasp. Rather than directly commanding joint angles or the absolute pose of the end effector in the world coordinate frame, each action specifies a relative displacement of the tool center point (TCP) or center of the gripper with respect to the pose of the target object. In other words, the action represents a position and orientation of the gripper relative to the currently selected item. In this formulation, the sole aim of the policy is to learn gripper configurations that lead to successful grasps, without reasoning about the

dynamics or planning of the robotic arm. To fully capture the 6D pose of the gripper in 3D space, the action is defined as a 6-dimensional, continuous vector:

$$\mathbf{a} = [\Delta l, \Delta w, \Delta h, \Delta \varphi, \Delta \omega, \Delta \psi],$$

where $\Delta l, \Delta w, \Delta h$ denote relative Cartesian offsets in meters along the object’s local x, y , and z axes, respectively, and $\Delta \varphi, \Delta \omega, \Delta \psi$ represent the relative rotations about the x, y , and z axes (roll, pitch, and yaw), respectively.

To execute an action, the agent first computes the absolute pose with component-wise addition between the relative action $\mathbf{a}$ and the preserved, true context $\mathbf{x}$. Once the absolute pose is determined, the agent uses an inverse kinematics (Calzada-Garcia et al., 2025) solver to compute a feasible joint configuration for the target pose. If a valid solution exists, a motion planner generates a trajectory that moves the manipulator from its starting state to the desired grasping position. If no valid solution is found, the action is considered infeasible and assigned a reward of zero.

For example, the action $[0.0, 0.0, 0.0, \frac{\pi}{4}, 0.0, 0.0]$ moves the TCP to the object’s centroid with matching pitch and yaw, while introducing a relative roll offset of $\frac{\pi}{4}$ radians or 45° . Figure 5.6 illustrates the object’s local axes and provides visual clarification of how each action component maps to translations and rotations about these axes. Because all components of the action are defined in the object’s local frame, the same relative action can produce different absolute motions depending on the object’s pose in the workspace. This introduces another element of stochasticity into the task, as the same action in the same context may lead to different results.

To ensure that the robot only attempts potentially successful grasps, each component of the relative action $\mathbf{a}$ has the following bounds

$$\begin{aligned} \Delta l &\in [-0.10, 0.10], \Delta w \in [-0.10, 0.10], \Delta h \in [-0.01, 0.10], \\ \Delta \varphi &\in [-\frac{\pi}{4}, \frac{\pi}{4}], \Delta \omega \in [-\frac{\pi}{4}, \frac{\pi}{4}], \Delta \psi \in [-\frac{\pi}{4}, \frac{\pi}{4}]. \end{aligned}$$

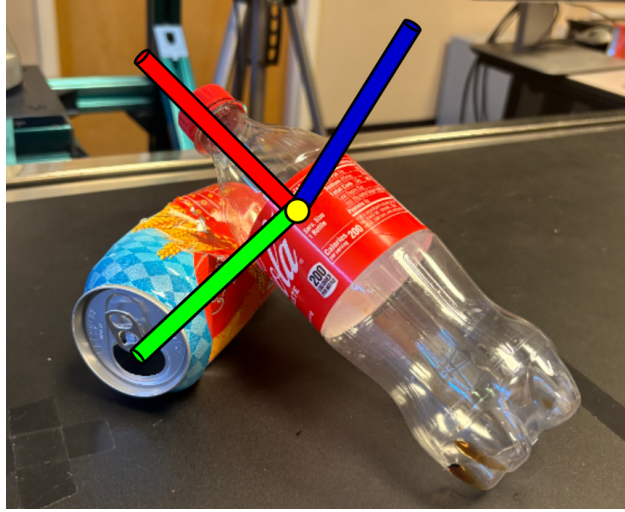


Figure 5.6: Local Coordinate Frame. The yellow dot on the bottle represents its centroid, while the red, green, and blue axes indicate the positive directions of the object's local x , y , and z axes, respectively. An action containing a $+\Delta l$ component would produce a translation along the red axis, starting from the yellow dot.

These bounds not only prevent the agent from proposing actions that will never lead to success, but they also shrink the action space significantly. In addition to these restrictions on the relative actions, the computed absolute pose is also clamped to the same bounds defined for the context (Section 5.2.3).

5.2.5 Reward Definition

Rewards are assigned autonomously to each action executed in the workspace. The reward is binary: actions that result in a successful grasp receive a reward of 1, while unsuccessful actions receive a reward of 0. Action success is determined using two criteria: a reduction in the total workspace items, and the measured current load of the gripper. If the number of items in the workspace decreases by one following the action and the gripper current exceeds 150 mA, the action is classified as successful and assigned a reward of 1. Otherwise, the action is classified as a failure and assigned a reward of 0. This dual-condition check ensures that grasp attempts that fail to grab the object are not incorrectly labeled as successful.

A binary reward setup is effective for evaluating the method because multiple actions can yield successful grasps and achieve the maximum reward. However, many of these actions are suboptimal due to inherent uncertainty in the workspace and the grasping process. Rather than optimizing for any action that simply achieves success, the modified objective encourages convergence towards actions that are more optimal and robust to uncertainty. These optimal actions are often located near the center of regions with consistently high rewards. Converging to these regions improves overall performance and helps avoid policies that settle in local optima or rely on highly situational actions.

5.2.6 Neural Network Architectures

Each approximate reward model in the ensemble is a fully connected Multi-Layer Perceptron (MLP). Each network has two hidden layers consisting of 256 units, followed by Rectified Linear Unit (ReLU) activations. The model takes the concatenated context and action and outputs a scalar value representing the predicted expected reward. The policy network is also a fully connected MLP with two hidden layers of 256 units. However, it uses Gaussian Error Linear Unit (GELU) activations after each hidden layer. The policy is initialized with Xavier weight initialization to produce larger variance in the network at first. Because the policy maps contexts to actions, a hyperbolic tangent (tanh) activation is applied to the final layer to bound the outputs within a normalized action range.

5.3 Baselines

In order to evaluate the proposed PAR objective, two robust RL baselines were implemented. Much like the PAR objective, these baselines focus on robustness to continuous action perturbations, making them a useful comparison. Although these methods were designed for MDPs or stochastic policies, the objectives of these methods were adapted for the approach in this paper. The two baselines are as follows:

- **Noisy Action Robust MDP (NRMDP):** The NRMDP (Tessler et al., 2019) method introduces a mixed policy π_{mix} which combines the agent’s policy π and the adversary’s policy $\bar{\pi}$ such that

$$a_t \sim \pi_{\text{mix}}(\pi(s_t), \bar{\pi}(s_t)).$$

The mixed policy is defined as

$$\pi_{\text{mix}}(a|s) = \mathbb{E}_{b \sim \pi(\cdot|s), \bar{b} \sim \bar{\pi}(\cdot|s)} \left[\mathbb{1}_{a=(1-\epsilon)b+\epsilon\bar{b}} \right],$$

where ϵ (denoted α in the original paper) is the adversarial coefficient. The corresponding optimization objective is

$$\pi_{\text{mix}}^* = \arg \max_{\pi} \min_{\bar{\pi}} \mathbb{E}_{a_t \sim \pi_{\text{mix}}} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$

In all experiments, the adversarial coefficient ϵ was fixed to a value of 0.10.

- **Adaptive Action Perturbation MDP (A2P):** The A2P (Liu et al., 2024) method extends the NRMDP by making the adversarial coefficient ϵ adaptive. At each iteration, the Euclidian distance between the agent and adversarial actions is computed as d . A moving average of the distance is maintained with the following:

$$d_j = \zeta \cdot d_{j-1} + (1 - \zeta) \cdot d,$$

where ζ (denoted β in the original paper) is the momentum factor. They then define a metric (unnamed) as

$$b = \text{sgn}(d_{j-1} - d) \sigma(|d_{j-1} - d|).$$

Finally, the adversarial coefficient ϵ is updated by

$$\epsilon_j = \epsilon_{j-1} + c \cdot b,$$

where c is the learning rate for ϵ . By adapting the adversarial coefficient, the method adjusts the strength of the adversarial perturbation to match the environment. In all experiments, the A2P objective was trained with a momentum factor ζ of 0.50 and an adversarial learning rate c of 0.10.

5.4 Experiments and Results

This section describes the evaluations of the proposed method across the simulation and physical settings and presents the corresponding results. First, the behavior of the different optimization objectives is analyzed in the Buffalo Gym environment, focusing on performance under different noise types. Then, a sensitivity analysis on the uncertainty coefficient β is conducted, also in the Buffalo Gym environment, to assess the stability of the approach. And finally, the different optimization objectives are evaluated on the robotic manipulation task, highlighting their effectiveness in a real-world grasping scenario. The experiments demonstrate that the proposed objective is effective and consistently outperforms other robust RL baseline methods.

5.4.1 Objective Comparison in Simulation

First, the different optimization objectives were analyzed in the Buffalo Gym environment. Instead of optimizing with an approximate reward model as described in the proposed framework, the Buffalo Gym functions were treated as the true deterministic reward functions during these experiments, allowing the behavior of each objective to be evaluated. Optimization was performed on the five predefined reward functions (Figure 5.1), with five different initialization seeds per function. After convergence, each policy was evaluated based on the true achieved reward and the average reward under three types of action noise: Gaussian noise with standard deviations equal to $\frac{1}{3}$ and $\frac{2}{3}$ of the maximum perturbation $\delta_{\max}$, and uniformly random noise within the noise interval. The results for each function across the five seeds are shown in Tables 5.2, 5.3, 5.4, 5.5, and 5.6. Additionally, a convergence example for each function is shown in Figure 5.7.

As expected, the standard RL objective ($\alpha = 0.0$) achieves the highest reward in the absence of perturbations across all five functions. Because this objective solely maximizes reward, it naturally converges to actions at the reward maxima. However, the PAR objective demonstrates superior performance under action noise in every scenario except for two (see Tables 5.4 and 5.5). This is a result of the performance/robustness tradeoff of the PAR objective, where the resulting actions are high reward and resilient to action perturbations. The optimal value of α depends on the magnitude of the noise, with larger perturbations favoring higher values of α . This behavior is intuitive, as noisier scenarios require more robust actions. Both A2P and NRMDP show weaker robustness performance in nearly all scenarios when compared to the PAR objective. Since they are optimized with respect to adversarial noise in one direction, the resulting actions may be sensitive to noise in the opposite direction.

Table 5.2: Objective Comparison Results for Function 1.

Objective	Reward ($\sigma = 0$)	$\sigma = \delta_{\max}/3$	$\sigma = 2\delta_{\max}/3$	Random
Standard ($\alpha = 0.0$)	0.7867 $\pm$ 0.00	0.6794 $\pm$ 0.00	0.6240 $\pm$ 0.00	0.5641 $\pm$ 0.00
PAR ($\alpha = 0.35$)	0.7807 $\pm$ 0.00	0.6887 $\pm$ 0.00	0.6395 $\pm$ 0.00	0.5884 $\pm$ 0.00
PAR ($\alpha = 0.50$)	0.7640 $\pm$ 0.00	0.6943 $\pm$ 0.00	0.6526 $\pm$ 0.00	0.6101 $\pm$ 0.00
PAR ($\alpha = 0.65$)	0.6919 $\pm$ 0.00	0.6931 $\pm$ 0.00	0.6776 $\pm$ 0.00	0.6623 $\pm$ 0.00
PAR ($\alpha = 1.0$)	0.6207 $\pm$ 0.00	0.6853 $\pm$ 0.00	0.6888 $\pm$ 0.00	0.6984 $\pm$ 0.00
A2P	0.7820 $\pm$ 0.00	0.6795 $\pm$ 0.01	0.6258 $\pm$ 0.01	0.5679 $\pm$ 0.02
NRMDP	0.7743 $\pm$ 0.01	0.6650 $\pm$ 0.02	0.6081 $\pm$ 0.02	0.5461 $\pm$ 0.03

Table 5.3: Objective Comparison Results for Function 2.

Objective	Reward ($\sigma = 0$)	$\sigma = \delta_{\max}/3$	$\sigma = 2\delta_{\max}/3$	Random
Standard ($\alpha = 0.0$)	0.5486 $\pm$ 0.05	0.4639 $\pm$ 0.04	0.4408 $\pm$ 0.06	0.4255 $\pm$ 0.08
PAR ($\alpha = 0.35$)	0.5030 $\pm$ 0.00	0.5039 $\pm$ 0.00	0.5067 $\pm$ 0.00	0.5176 $\pm$ 0.00
PAR ($\alpha = 0.50$)	0.4967 $\pm$ 0.00	0.5054 $\pm$ 0.00	0.4966 $\pm$ 0.00	0.5004 $\pm$ 0.00
PAR ($\alpha = 0.65$)	0.4950 $\pm$ 0.00	0.5062 $\pm$ 0.00	0.4957 $\pm$ 0.00	0.4918 $\pm$ 0.00
PAR ($\alpha = 1.0$)	0.4970 $\pm$ 0.00	0.5035 $\pm$ 0.00	0.4976 $\pm$ 0.00	0.4926 $\pm$ 0.01
A2P	0.5422 $\pm$ 0.05	0.4509 $\pm$ 0.05	0.4291 $\pm$ 0.08	0.4171 $\pm$ 0.09
NRMDP	0.3088 $\pm$ 0.24	0.4002 $\pm$ 0.12	0.3953 $\pm$ 0.12	0.3972 $\pm$ 0.11

Table 5.4: Objective Comparison Results for Function 3.

Objective	Reward ($\sigma = 0$)	$\sigma = \delta_{\max}/3$	$\sigma = 2\delta_{\max}/3$	Random
Standard ($\alpha = 0.0$)	0.5411 $\pm$ 0.00	0.4779 $\pm$ 0.03	0.4565 $\pm$ 0.05	0.4464 $\pm$ 0.07
PAR ($\alpha = 0.35$)	0.5406 $\pm$ 0.00	0.4983 $\pm$ 0.00	0.4955 $\pm$ 0.00	0.5013 $\pm$ 0.00
PAR ($\alpha = 0.50$)	0.5396 $\pm$ 0.00	0.4982 $\pm$ 0.00	0.4956 $\pm$ 0.00	0.5014 $\pm$ 0.00
PAR ($\alpha = 0.65$)	0.5360 $\pm$ 0.01	0.4979 $\pm$ 0.00	0.4957 $\pm$ 0.00	0.5013 $\pm$ 0.00
PAR ($\alpha = 1.0$)	0.4536 $\pm$ 0.00	0.4937 $\pm$ 0.00	0.4964 $\pm$ 0.00	0.4916 $\pm$ 0.00
A2P	0.5381 $\pm$ 0.00	0.4985 $\pm$ 0.00	0.4946 $\pm$ 0.00	0.4993 $\pm$ 0.00
NRMDP	0.5386 $\pm$ 0.00	0.4984 $\pm$ 0.00	0.4950 $\pm$ 0.00	0.5001 $\pm$ 0.00

Table 5.5: Objective Comparison Results for Function 4.

Objective	Reward ($\sigma = 0$)	$\sigma = \delta_{\max}/3$	$\sigma = 2\delta_{\max}/3$	Random
Standard ($\alpha = 0.0$)	0.5560 $\pm$ 0.07	0.4826 $\pm$ 0.01	0.4465 $\pm$ 0.05	0.4194 $\pm$ 0.07
PAR ($\alpha = 0.35$)	0.5526 $\pm$ 0.07	0.4794 $\pm$ 0.01	0.4440 $\pm$ 0.04	0.4174 $\pm$ 0.07
PAR ($\alpha = 0.50$)	0.5495 $\pm$ 0.07	0.4767 $\pm$ 0.01	0.4417 $\pm$ 0.04	0.4153 $\pm$ 0.07
PAR ($\alpha = 0.65$)	0.5462 $\pm$ 0.08	0.4743 $\pm$ 0.01	0.4396 $\pm$ 0.04	0.4136 $\pm$ 0.07
PAR ($\alpha = 1.0$)	0.4741 $\pm$ 0.00	0.4667 $\pm$ 0.00	0.4620 $\pm$ 0.00	0.4585 $\pm$ 0.00
A2P	0.5016 $\pm$ 0.01	0.4745 $\pm$ 0.03	0.4569 $\pm$ 0.05	0.4418 $\pm$ 0.07
NRMDP	0.4603 $\pm$ 0.08	0.4561 $\pm$ 0.07	0.4465 $\pm$ 0.07	0.4380 $\pm$ 0.08

Table 5.6: Objective Comparison Results for Function 5.

Objective	Reward ($\sigma = 0$)	$\sigma = \delta_{\max}/3$	$\sigma = 2\delta_{\max}/3$	Random
Standard ($\alpha = 0.0$)	0.4552 $\pm$ 0.03	0.4047 $\pm$ 0.02	0.3888 $\pm$ 0.03	0.3728 $\pm$ 0.04
PAR ($\alpha = 0.35$)	0.4546 $\pm$ 0.03	0.4066 $\pm$ 0.02	0.3908 $\pm$ 0.02	0.3748 $\pm$ 0.04
PAR ($\alpha = 0.50$)	0.4529 $\pm$ 0.03	0.4083 $\pm$ 0.02	0.3927 $\pm$ 0.02	0.3769 $\pm$ 0.04
PAR ($\alpha = 0.65$)	0.4442 $\pm$ 0.03	0.4119 $\pm$ 0.01	0.4045 $\pm$ 0.00	0.3990 $\pm$ 0.01
PAR ($\alpha = 1.0$)	0.3994 $\pm$ 0.00	0.3984 $\pm$ 0.00	0.4062 $\pm$ 0.00	0.4163 $\pm$ 0.00
A2P	0.4514 $\pm$ 0.03	0.4027 $\pm$ 0.02	0.3875 $\pm$ 0.03	0.3719 $\pm$ 0.04
NRMDP	0.4449 $\pm$ 0.03	0.3958 $\pm$ 0.03	0.3813 $\pm$ 0.04	0.3663 $\pm$ 0.05

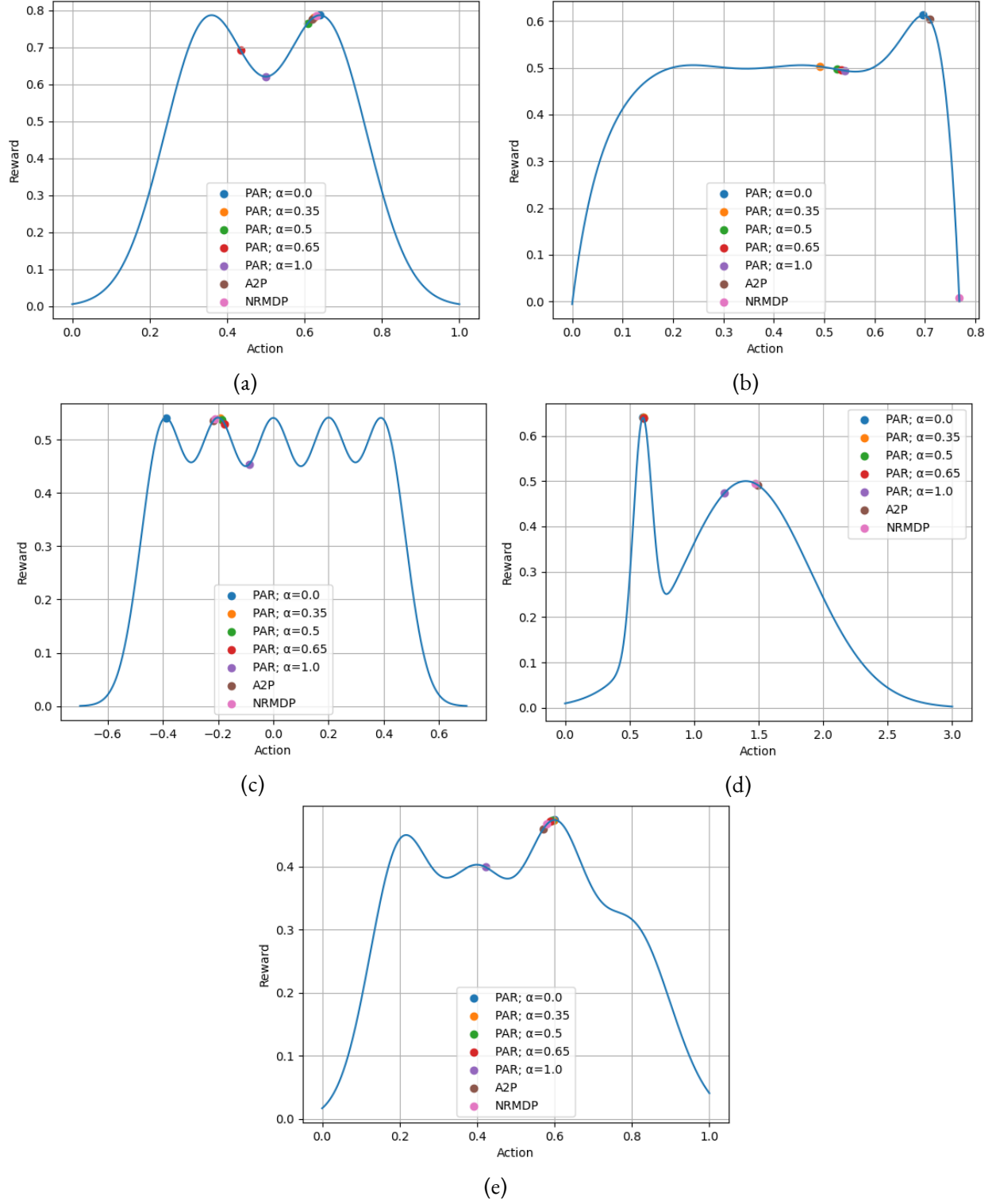


Figure 5.7: Convergence Examples from Objective Comparison. Each figure above represents a single-seed example of each objectives' resulting action plotted on the true reward function for each of the five Buffalo Gym functions.

5.4.2 Sensitivity Analysis in Simulation

Next, a sensitivity analysis on the uncertainty coefficient β from Equation 4.3 was performed in the Buffalo Gym environment. In these experiments, the Buffalo Gym functions were interpreted as stochastic reward functions, and binary rewards were sampled based on the returned reward probability. After generating 300 random samples from the reward function, an ensemble of 5 approximate reward models ($N = 5$) was trained. To encourage diversity in the ensemble, gaps were introduced into the sampling distribution. Policies were then trained using different values of β , with each policy using the PAR objective with $\alpha = 0.5$. After training, the converged policies were evaluated using the true expected reward function to measure the actual achieved performance. The analysis was performed on three reward functions with three randomization seeds each. The results for each function across the three seeds are shown in Tables 5.7, 5.8, and 5.9. An example of the resulting reward models with varying values of β for each function is shown in Figures 5.8, 5.9, and 5.10.

In all three scenarios, the true achieved reward remains relatively stable across a broad range of β values, indicating that the method is not sensitive to the uncertainty coefficient. However, when β is less than 2.0, the approximate reward consistently overestimates the true reward, which can lead to errors in the converged policy. Depending on the desired conservativeness of the reward model, it seems best to choose values of β greater than or equal to 2.0.

Table 5.7: Function 1 Sensitivity Analysis Results.

β	True Reward	Approx. Reward
0.0	0.6814 ± 0.04	0.7560 ± 0.06
1.0	0.7055 ± 0.06	0.7217 ± 0.04
2.0	0.7137 ± 0.07	0.6957 ± 0.04
3.0	0.6767 ± 0.05	0.6150 ± 0.06
4.0	0.6720 ± 0.05	0.5909 ± 0.06
5.0	0.6690 ± 0.05	0.5692 ± 0.07

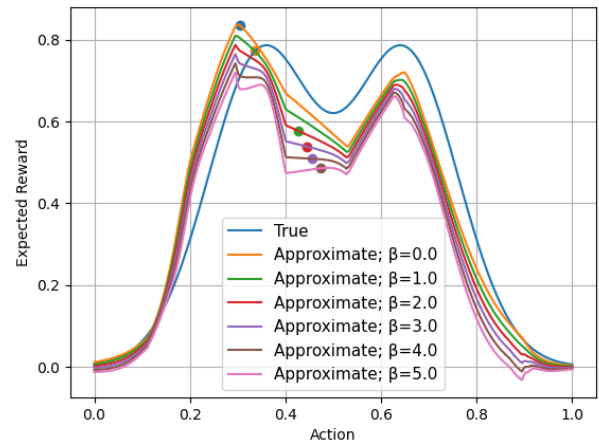


Figure 5.8: Function 1 Sensitivity Analysis.

Table 5.8: Function 4 Sensitivity Analysis Results.

β	True Reward	Approx. Reward
0.0	0.4953 ± 0.01	0.5084 ± 0.04
1.0	0.4960 ± 0.01	0.4973 ± 0.03
2.0	0.4962 ± 0.00	0.4867 ± 0.03
3.0	0.4965 ± 0.00	0.4759 ± 0.02
4.0	0.4966 ± 0.00	0.4655 ± 0.02
5.0	0.4965 ± 0.00	0.4551 ± 0.01

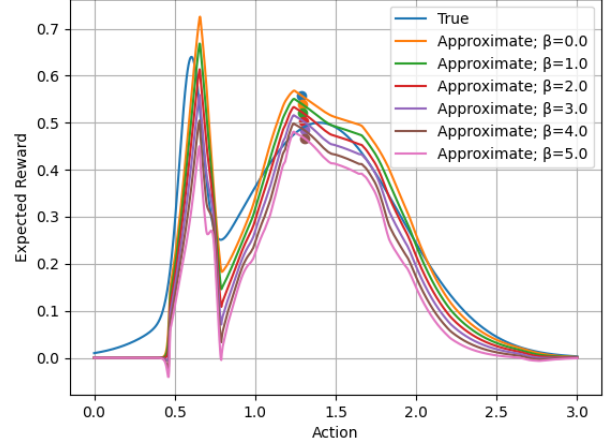


Figure 5.9: Function 4 Sensitivity Analysis.

Table 5.9: Function 5 Sensitivity Analysis Results.

β	True Reward	Approx. Reward
0.0	0.4259 ± 0.03	0.4587 ± 0.08
1.0	0.4255 ± 0.03	0.4416 ± 0.08
2.0	0.4144 ± 0.03	0.4188 ± 0.08
3.0	0.4129 ± 0.02	0.4111 ± 0.06
4.0	0.4187 ± 0.02	0.3683 ± 0.03
5.0	0.4222 ± 0.02	0.3561 ± 0.03

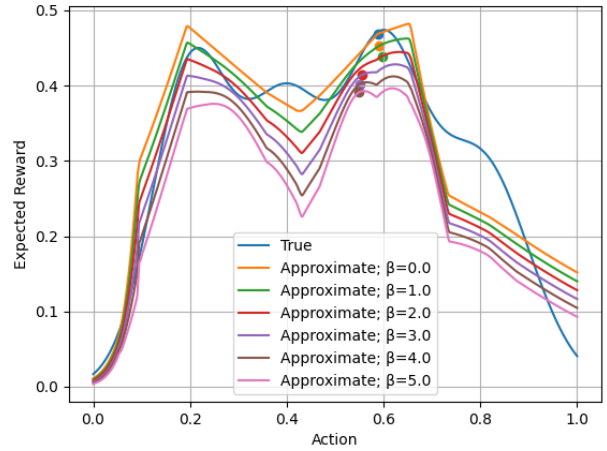


Figure 5.10: Function 5 Sensitivity Analysis.

5.4.3 Objective Comparison on Robotic Task

Finally, the different objectives were evaluated on the physical sorting task described in Section 5.2, in which a robotic manipulator has to grasp and sort recyclable objects. An initial dataset consisting of 160 expert and 340 random demonstrations was first collected. The expert demonstrations were generated by hand-guiding the KUKA collaborative robot to a variety of feasible grasp poses for the assigned object, while the random demonstrations were generated using an exploration policy that sampled actions uniformly within the action bounds. An expected reward ensemble consisting of 5 networks ($N = 5$) was then trained on

the initial dataset. The same expected reward model was subsequently used to train each objective across 8 initialization seeds with $\beta = 2.0$. The learned policies were evaluated on 3 workspace configurations containing a total of 5 bottles and 5 cans. For every policy, 10 grasps were executed on the 3 workspace configurations without action perturbations, followed by 10 grasps with adversarial action perturbations of ± 2 cm in each translational direction and $\pm 6^\circ$ in each rotational direction. Across the 8 initialization seeds, this amounts to 80 grasp attempts without noise and 80 grasp attempts with perturbations for each objective, resulting in a total of 800 grasp attempts across the entire experiment. The grasp success rates across all seeds are shown in Table 5.10, while the per-seed success rates without noise, with noise, and combined are shown in Tables 5.11 5.12, and 5.13.

Table 5.10: Grasp Success Rates from Robotic Sorting Task.

Objective	Total Success Rate		
	Without Noise	With Noise	Overall
Standard ($\alpha = 0$)	70.0% (56/80)	31.3% (25/80)	50.6% (81/160)
PAR ($\alpha = 0.5$)	95.0% (76/80)	75.0% (60/80)	85.0% (136/160)
PAR ($\alpha = 1.0$)	85.0% (68/80)	61.3% (49/80)	73.1% (117/160)
A2P	86.3% (69/80)	57.5% (46/80)	71.9% (115/160)
NRMDP	67.5% (54/80)	37.5% (30/80)	52.5% (84/160)

The PAR objective with $\alpha = 0.5$ achieved the best performance in both settings, executing 19 more successful grasps than the second best objective and 55 more successful grasps than the worst performing method. This result is consistent with the intended trade-off between performance and robustness, leading to high reward actions within high reward regions. Notably, standard RL fails to achieve the highest success rate even in the noiseless setting. This suggests that the PAR objective may be less prone to reward model exploitation, as it chooses wide regions rather than pure maxima that could be misestimated. The PAR objective with $\alpha = 1.0$ achieved the third highest success rate without noise, but the second highest success rate with noise. This behavior matches the increased emphasis on robustness, potentially leading the policy to select actions that do not achieve the optimal reward but are located in regions where performance is

less sensitive to perturbations. Although NRMDP and A2P showed improved robustness compared to standard RL, these methods remain vulnerable to inaccuracies in the learned reward model.

Table 5.11: Complete Grasp Success Rates for Actions without Noise.

Objective	Successful Grasps per Seed								Total Success Rate
	S ₁	S ₂	S ₃	S ₄	S ₅	S ₆	S ₇	S ₈	
Standard ($\alpha = 0.0$)	8	9	10	4	8	2	6	9	70.0% (56/80)
PAR ($\alpha = 0.5$)	10	10	9	10	10	10	9	8	95.0% (76/80)
PAR ($\alpha = 1.0$)	10	9	10	2	10	10	8	9	85.0% (68/80)
A2P	9	9	10	10	10	6	10	5	86.3% (69/80)
NRMDP	9	8	10	6	6	1	9	5	67.5% (54/80)

Table 5.12: Complete Grasp Success Rates for Actions with Noise.

Objective	Successful Grasps per Seed								Total Success Rate
	S ₁	S ₂	S ₃	S ₄	S ₅	S ₆	S ₇	S ₈	
Standard ($\alpha = 0.0$)	3	4	6	1	5	0	3	3	31.3% (25/80)
PAR ($\alpha = 0.5$)	8	8	4	9	9	9	5	8	75.0% (60/80)
PAR ($\alpha = 1.0$)	8	7	7	0	9	9	3	6	61.3% (49/80)
A2P	8	7	5	6	7	6	6	1	57.5% (46/80)
NRMDP	7	4	8	3	3	0	3	2	37.5% (30/80)

Table 5.13: Complete Grasp Success Rates for All Actions.

Objective	Successful Grasps per Seed								Total Success Rate
	S ₁	S ₂	S ₃	S ₄	S ₅	S ₆	S ₇	S ₈	
Standard ($\alpha = 0.0$)	11	13	16	5	13	2	9	12	50.6% (81/160)
PAR ($\alpha = 0.5$)	18	18	13	19	19	19	14	16	85.0% (136/160)
PAR ($\alpha = 1.0$)	18	16	17	2	19	19	11	15	73.1% (117/160)
A2P	17	16	15	16	17	12	16	6	71.9% (115/160)
NRMDP	16	12	18	9	9	1	12	7	52.5% (84/160)

CHAPTER 6

CONCLUSION

The application of reinforcement learning to real-world problems remains challenging. As discussed earlier, several approaches, including robust, model-based, and offline reinforcement learning, look to overcome these difficulties in a variety of ways. Whether the goal is to increase sample efficiency through learned models, reduce the cost of data collection with offline optimization, or account for uncertainty during the learning process, these methods help mitigate many of the problems associated with physical deployment. However, each approach also introduces its own limitations and trade-offs.

The proposed approach combines ideas from each of these methods, aiming to leverage their strengths while compensating for their weaknesses. First, a data collection strategy is employed to build an initial dataset of context-action-reward tuples. This strategy incorporates samples from an expert policy and an exploration policy, combining the high-reward behavior of the expert with the broad action coverage provided by exploration to produce a diverse dataset. Then, a reward model is trained on the data, represented by an ensemble of networks. The ensemble outputs reward predictions while incorporating an uncertainty penalty, producing more conservative reward estimates in regions where the model is less certain. Next, a robust optimization objective, named the PAR objective, is used to optimize a policy. This objective produces policies that trade off robustness and performance, leading to high-reward actions that remain resilient to perturbations. Finally, the method integrates iterative policy deployment and data aggregation, followed by retraining of the reward model and policy. This process improves model accuracy

in relevant regions and yields better policies over time. Together, these components provide a practical framework for applying reinforcement learning in real-world settings.

To evaluate the proposed method, experiments were conducted in simulated and physical settings against two robust reinforcement learning baselines. In the Buffalo Gym environment, the objective comparison showed that the PAR objective achieved the best performance under noisy conditions. Furthermore, the sensitivity analysis demonstrated that the approach remains stable across a range of values for the uncertainty coefficient β . Finally, the objectives were evaluated on the robotic sorting task, where the PAR objective again outperformed all methods across all scenarios. Collectively, these results demonstrate the effectiveness of the proposed approach and its ability to address the challenges of real-world reinforcement learning.

6.1 Limitations

Although this method provides an effective framework for real-world reinforcement learning, several limitations remain. Even with the proposed data collection strategy, acquiring real-world data is still expensive and time consuming. As a result, obtaining sufficient data becomes increasingly challenging as tasks become more complex. Additionally, while the use of an approximate reward model is more realistic than assuming access to the true reward function, model inaccuracies can negatively affect performance. Because the policy is optimized with respect to the learned reward model, prediction errors may compound during optimization and lead to suboptimal behavior. In the robotic setup, the discretized context simplifies learning by drastically reducing the size of the context space, but it also provides an incomplete representation of the environment. Important factors such as the number of items, object deformation and occlusion are omitted, limiting the agent’s ability to fully understand the task. Finally, the contextual bandit formulation has no notion of time and instead focuses solely on the immediate reward. This shrinks the application of the approach to relatively simple settings, as it cannot account for the effects of current actions on future rewards.

6.2 Future Work

The work in this paper has the potential to be extended in several future directions. First, it would be valuable to extend this approach from the contextual bandit setting to the Markov decision process framework. This would allow the method to be applied to more complex sequential decision-making tasks where long-term consequences must be considered. Additionally, incorporating the robotic setup into a simulation environment like Gazebo or MuJoCo would allow experimentation with larger state and action spaces and enable the use of more data-intensive learning methods. Transfer learning techniques could then be used to deploy policies learned in simulation on the real robot. Finally, future work could investigate the use of stochastic policies rather than deterministic ones. This could lead to improved exploration during training and the development of more adaptable policies.

APPENDIX A

THEORETICAL JUSTIFICATION FOR UNCERTAINTY PENALIZATION

Theorem 1 (Probabilistic Bound of Reward Model). *Suppose the ensemble networks estimation method has a true mean $\mathcal{R}(x, a) = \mathbb{E}[\bar{\mathcal{R}}_{\theta_i}(x, a)]$ and common variance $\sigma^2 = \text{Var}(\bar{\mathcal{R}}_{\theta_i}), \forall i$. Then, the uncertainty-penalized reward model $\hat{\mathcal{R}}_{\theta}(x, a) = \mu_{\theta}(x, a) - \beta\sigma_{\theta}(x, a)$ (Eq. 4.3) satisfies the following with probability $\geq 1 - \delta$ for any $\delta \in (0, 1)$ and $\beta \geq \sqrt{\frac{1}{N\delta}}$*

$$\hat{\mathcal{R}}_{\theta}(x, a) \leq \mathcal{R}(x, a)$$

Proof. Consider the probability that the estimate μ_{θ} deviates from the true reward $\mathcal{R}$ by more than $\beta\sigma_{\theta}$:

$$\mathbb{P}[\mathcal{R}(x, a) < \mu_{\theta}(x, a) - \beta\sigma_{\theta}(x, a)] \leq \mathbb{P}[|\mu_{\theta}(x, a) - \mathcal{R}(x, a)| > \beta\sigma_{\theta}(x, a)]$$

Applying Chebychev's inequality to the random variable μ_{θ} :

$$\mathbb{P}(|\mu_{\theta}(x, a) - \mathcal{R}(x, a)| \geq \epsilon) \leq \frac{\text{Var}(\mu_{\theta})}{\epsilon^2}$$

Setting $\epsilon = \beta\sigma_\theta(x, a)$ and noting that $\text{Var}(\mu_\theta) = \sigma^2/N$:

$$\mathbb{P}(|\mu_\theta(x, a) - \mathcal{R}(x, a)| \geq \beta\sigma_\theta(x, a)) \leq \frac{\sigma^2/N}{(\beta\sigma_\theta(x, a))^2}$$

Under the assumption that the ensemble standard deviation $\sigma_\theta(x, a)$ is an unbiased estimator of the population standard deviation σ (i.e., $\mathbb{E}[\sigma_\theta^2] = \sigma^2$), we have:

$$\mathbb{P}(|\mu_\theta(x, a) - \mathcal{R}(x, a)| \geq \beta\sigma_\theta(x, a)) \leq \frac{\sigma^2}{N\beta^2\sigma^2} = \frac{1}{N\beta^2}$$

To ensure the probability of the lower bound being violated is at most δ , we set:

$$\frac{1}{N\beta^2} \leq \delta \implies \beta^2 \geq \frac{1}{N\delta} \implies \beta \geq \sqrt{\frac{1}{N\delta}}$$

Thus, with probability at least $1 - \delta$, $\mu_\theta(x, a) - \beta\sigma_\theta(x, a) \leq \mathcal{R}(x, a)$, i.e., $\hat{\mathcal{R}}_\theta(x, a) \leq \mathcal{R}(x, a)$. $\square$

Thus with a high probability, the uncertainty-penalized reward model does not overestimate the true reward even in underexplored regions.

APPENDIX B

THEORETICAL ANALYSIS OF CONVERGENCE

To analyze the convergence of the robust policy gradient, we first establish the smoothness properties of the Perturbed Action Robustness (PAR) function and the resulting objective function.

Lemma 1 (Smoothness of PAR Function). *Assume the approximate reward model $\hat{\mathcal{R}}(x, a)$ (as defined in Sec. 4.1.2) is twice continuously differentiable w.r.t. action a , i.e., $\hat{\mathcal{R}}$ and $\nabla_a \hat{\mathcal{R}}$ are Lipschitz continuous with constants L_{R_1} and L_{R_2} respectively. Then the PAR function (defined in Eq. 4.4) is also twice continuously differentiable w.r.t. a , i.e., $\eta(x, a)$ and $\nabla_a \eta(x, a)$ are also Lipschitz continuous with constants L_{R_1} and L_{R_2} respectively. In particular, the gradient of $\eta(x, a)$ is bounded.*

Proof. Using the definition of the PAR function (Eq. 4.4) and taking its gradient with respect to a :

$$\nabla_a \eta(x, a) = \frac{1}{Z} \int_{-\delta_{\max}}^{\delta_{\max}} \nabla_a \hat{\mathcal{R}}(x, a + \delta) d\delta$$

Applying the triangle inequality for integrals and the Lipschitz assumption $\|\nabla_a \hat{\mathcal{R}}\| \leq L_{R_1}$:

$$\|\nabla_a \eta(x, a)\| \leq \frac{1}{Z} \int_{-\delta_{\max}}^{\delta_{\max}} \|\nabla_a \hat{\mathcal{R}}(x, a + \delta)\| d\delta \leq \frac{1}{Z} \int_{-\delta_{\max}}^{\delta_{\max}} L_{R_1} d\delta = \frac{2\delta_{\max} L_{R_1}}{Z}$$

Given that $Z = 2\delta_{\max}$ (from Eq. 4.5), we have:

$$\|\nabla_a \eta(x, a)\| \leq L_{R_1}$$

This proves that η is L_{R_1} -Lipschitz and that its gradient is bounded by a constant independent of a and x , preventing gradient explosion during optimization.

Now consider the difference between gradients of η at a and a' :

$$\begin{aligned} \|\nabla_a \eta(x, a') - \nabla_a \eta(x, a)\| &= \left\| \frac{1}{Z} \int_{-\delta_{\max}}^{\delta_{\max}} \left(\nabla_a \hat{\mathcal{R}}(x, a' + \delta) - \nabla_a \hat{\mathcal{R}}(x, a + \delta) \right) d\delta \right\| \\ &\leq \frac{1}{Z} \int_{-\delta_{\max}}^{\delta_{\max}} \|\nabla_a \hat{\mathcal{R}}(x, a' + \delta) - \nabla_a \hat{\mathcal{R}}(x, a + \delta)\| d\delta \\ &\leq L_{R_2} \|a' - a\| \end{aligned}$$

Therefore, $\nabla_a \eta$ is also Lipschitz continuous with constant L_{R_2} . □

Lemma 2 (L-Smoothness of the Robust Objective). *Assume the policy $\pi_\phi(x)$ is twice continuously differentiable, i.e., $\pi_\phi(x)$ and its gradient $\nabla_\phi \pi_\phi(x)$ are L_{π_1} - and L_{π_2} -Lipschitz continuous respectively, with respect to ϕ . Further, from Lemma 1, $\nabla_a \hat{\mathcal{R}}$ and $\nabla_a \eta$ are both L_{R_2} -Lipschitz continuous in a . Then the objective function $J(\phi)$ defined in Eq. 4.6 is L -smooth, i.e., its gradient $\nabla J(\phi)$ is L -Lipschitz continuous:*

$$\|\nabla J(\phi_1) - \nabla J(\phi_2)\| \leq L \|\phi_1 - \phi_2\|$$

where

$$L = L_{\pi_2} L_{R_1} + L_{\pi_1} L_{R_2} \tag{B.1}$$

Proof. The gradient of the objective is given by Eq. 4.7:

$$\nabla_\phi J(\phi) = \mathbb{E}_{x \sim \rho_x} [\nabla_\phi \pi_\phi(x) \cdot \Gamma(x, \pi_\phi(x))]$$

where $\Gamma(x, a) = \nabla_a((1 - \alpha) \cdot \hat{\mathcal{R}}(x, a) + \alpha \cdot \eta(x, a))$. The difference between gradients at ϕ_1 and ϕ_2 is:

$$\nabla J(\phi_1) - \nabla J(\phi_2) = \mathbb{E}_{x \sim \rho_x} [(\nabla_{\phi} \pi_{\phi_1} - \nabla_{\phi} \pi_{\phi_2}) \Gamma(x, \pi_{\phi_1}) + \nabla_{\phi} \pi_{\phi_2} (\Gamma(x, \pi_{\phi_1}) - \Gamma(x, \pi_{\phi_2}))]$$

Using the Lipschitz property of $\nabla_{\phi} \pi_{\phi}$ and the boundedness of Γ (from Lemma 1), the first term is bounded by $M_1 \|\phi_1 - \phi_2\|$, where $M_1 = L_{\pi_2} L_{R_1}$. For the second term, since Γ is Lipschitz in a and π_{ϕ} is Lipschitz in ϕ , the difference $\|\Gamma(x, \pi_{\phi_1}) - \Gamma(x, \pi_{\phi_2})\|$ is bounded by $M_2 \|\phi_1 - \phi_2\|$, where $M_2 = L_{\pi_1} L_{R_2}$. Combining these, $\|\nabla J(\phi_1) - \nabla J(\phi_2)\| \leq L \|\phi_1 - \phi_2\|$ for L in Eq. B.1. $\square$

Theorem 2 (Convergence of Policy Parameters). *Consider the sequence of parameters $\{\phi_k\}$ generated by the gradient ascent update rule $\phi_{k+1} = \phi_k + \lambda \nabla J(\phi_k)$ (Eq. 3.7 applied in inner loop of Alg. 2). If the step size λ satisfies $0 < \lambda < \frac{2}{L}$ (L defined in Eq. B.1), then the sequence converges to a stationary point, i.e.,*

$$\lim_{k \rightarrow \infty} \|\nabla J(\phi_k)\| = 0$$

Proof. Since $J(\phi)$ is L -smooth (Lemma 2), we can apply the Descent Lemma (adapted for ascent):

$$J(\phi_{k+1}) \geq J(\phi_k) + \langle \nabla J(\phi_k), \phi_{k+1} - \phi_k \rangle - \frac{L}{2} \|\phi_{k+1} - \phi_k\|^2$$

Substituting the update rule $\phi_{k+1} - \phi_k = \lambda \nabla J(\phi_k)$:

$$J(\phi_{k+1}) \geq J(\phi_k) + \lambda \|\nabla J(\phi_k)\|^2 - \frac{L\lambda^2}{2} \|\nabla J(\phi_k)\|^2$$

$$J(\phi_{k+1}) - J(\phi_k) \geq \lambda \left(1 - \frac{L\lambda}{2}\right) \|\nabla J(\phi_k)\|^2$$

Summing from $k = 0$ to N :

$$J(\phi_{N+1}) - J(\phi_0) \geq \lambda \left(1 - \frac{L\lambda}{2}\right) \sum_{k=0}^N \|\nabla J(\phi_k)\|^2$$

If $\alpha \in [0, 1]$ and $\hat{\mathcal{R}}(x, a) \leq r_{\max} \forall(x, a)$, then $J(\phi) \leq r_{\max}$. Since $J(\phi)$ is bounded from above by $r_{\max}$, the left side is bounded:

$$\sum_{k=0}^{\infty} \|\nabla J(\phi_k)\|^2 \leq \frac{r_{\max} - J(\phi_0)}{\lambda(1 - \frac{L\lambda}{2})} < \infty$$

For the infinite series to converge, the general term must vanish:

$$\lim_{k \rightarrow \infty} \|\nabla J(\phi_k)\| = 0$$

Thus, the sequence $\{\phi_k\}$ converges to a stationary point of the robust objective in Eq. 4.6. □

APPENDIX C

THEORETICAL ANALYSIS OF ITERATIVE DEPLOYMENT STABILITY

To analyze the stability of the iterative deployment process described in Sec. 4.2.2, we investigate the risk of model collapse—where the policy π_ϕ converges to a narrow, suboptimal region of the action space due to a self-reinforcing feedback loop between the approximate reward model $\hat{\mathcal{R}}$ and the data collection process.

Assumption 1 (Distributional Coverage). *The initial dataset $\mathcal{D}_0$ (containing expert π_E and exploration π_β data) ensures that the initial state-action distribution $\rho_0(x, a)$ satisfies $\inf_{x,a} \rho_0(x, a) \geq \epsilon > 0$ for some small ϵ .*

Lemma 3 (Boundedness of the Density Ratio). *Let $\rho_{\pi_{\phi_i}}(x, a)$ be the state-action distribution induced by policy π_{ϕ_i} at iteration i . Under Assumption 1, the density ratio $w_i(x, a) = \frac{\rho_{\pi_{\phi_i}}(x, a)}{\rho_0(x, a)}$ is bounded such that $\sup_{x,a} w_i(x, a) \leq \frac{1}{\epsilon}$, preventing catastrophic distribution shift during iterative updates.*

Proof. From Sec. 4.2.2, the dataset $\mathcal{D}_{i+1}$ is the union of $\mathcal{D}_i$ and the on-policy data $\mathcal{D}_{\pi_{\phi_i}}$. The density of the training set at iteration i is $\rho_i = \frac{1}{i+1} \sum_{j=0}^i \rho_{\pi_{\phi_j}}$. Since ρ_0 is the distribution of $\mathcal{D}_0$, and $\rho_0(x, a) \geq \epsilon$ by Assumption 1, the ratio $\frac{\rho_{\pi_{\phi_i}}(x, a)}{\rho_0(x, a)}$ is bounded from above by $1/\epsilon$ because the probability density $\rho_{\pi_{\phi_i}}$ is

bounded by 1. Thus, the support of the reward model is always maintained, ensuring that the policy does not enter regions of zero data density where the model $\hat{\mathcal{R}}_\theta$ would be undefined. $\square$

Next, we note that during iterative deployment, the action executed is $a_t = \pi_{\phi_i}(x) + \xi$, where $\xi \sim \mathcal{N}(0, \sigma_{noise}^2)$.

Lemma 4 (Coverage Guarantee). *Given the additive Gaussian noise ξ used during the iterative deployment, for any action $a \in \mathcal{A}$ and context $x \in \mathcal{X}$, the probability of sampling action a given policy π_{ϕ_i} is strictly positive, i.e., $d^{\pi_{\phi_i}}(x, a) > 0$. Consequently, the dataset $D_{i+1} = D_i \cup D_{\pi_{\phi_i}}$ satisfies a condition of persistent excitation over the support of $\mathcal{A}$.*

Proof. The deployment policy is stochastic and defined by the density $p(a|x) = \frac{1}{\sqrt{2\pi\sigma_{noise}^2}} \exp\left(-\frac{(a-\pi_{\phi_i}(x))^2}{2\sigma_{noise}^2}\right)$. Since the Gaussian distribution has global support over $\mathbb{R}^n$, any $a \in \mathcal{A}$ has a non-zero probability of being sampled in every iteration i . Thus, the state-action visitation distribution $\rho_{D_i}(x, a)$ is bounded away from zero for all $a \in \mathcal{A}$, preventing the agent from becoming entirely blind to any region of the action space. $\square$

Now we evaluate the convergence of the policy parameters ϕ_i and the behavior of the gradient approximation error in Eq. 4.8.

Lemma 5 (Reward Model Uncertainty Bound). *Assuming the true expected reward function $\mathcal{R}$ is Lipschitz continuous and the underlying context-action distribution has bounded support, the expected error in the reward estimate for a dataset $\mathcal{D}$ is bounded by:*

$$\mathbb{E}_{(x,a) \sim \rho_{x,a}} \left[\left| \hat{\mathcal{R}}_\theta(x, a) - \mathcal{R}(x, a) \right| \right] \leq \mathcal{O} \left(\sqrt{\frac{\ln N}{N}} \right) + \beta \sigma_\theta(x, a), \quad (\text{C.1})$$

where σ_θ represents the empirical standard deviation of the ensemble predictions.

Proof. The error term consists of two components: the approximation error of the ensemble mean μ_θ and the penalization term $\beta \sigma_\theta$. Since each network $\bar{\mathcal{R}}_{\theta_i}$ is trained on independently sampled mini-batches

(Alg. 1, lines 4-5), by Hoeffding's inequality applied to the ensemble mean, the deviation of μ_θ from $\mathcal{R}$ decreases with N . The term $\beta\sigma_\theta$ explicitly bounds the uncertainty penalty. Combining these yields the result. $\square$

Given a dataset $\mathcal{D}_i$ with coverage over the context-action space, the uncertainty-penalized reward model $\hat{\mathcal{R}}_\theta$ satisfies $\lim_{\|\mathcal{D}_i\| \rightarrow \infty} \mathbb{E}_{x,a}[\|\hat{\mathcal{R}}_\theta(x, a) - \mathcal{R}(x, a)\|] = 0$, since the density of $\mathcal{D}_i$ is non-zero everywhere in the reachable action space by Lemma 4.

Lemma 6 (Gradient Approximation Error Convergence). *The error ϵ_i between the approximated gradient (Eq. 4.8) and the true gradient (Eq. 4.7) of the robust objective $J(\phi)$ vanishes as the dataset $\mathcal{D}_i$ grows and coverage is maintained:*

$$\mathbb{E}_{B \sim \mathcal{D}_i} \left[\|\nabla_\phi J(\phi) - \widehat{\nabla_\phi J}(\phi)\| \right] \leq C \left(\frac{1}{\sqrt{|B|}} + \|\hat{\mathcal{R}}_\theta - \mathcal{R}\|_\infty \right), \quad (\text{C.2})$$

where C is a constant dependent on the Lipschitz constants of π_ϕ and $\mathcal{R}$, and $|B|$ is the batch size sampled from $\mathcal{D}_i$.

Proof. The gradient approximation in Eq. 4.8 is a Monte Carlo estimate using the learned reward model $\hat{\mathcal{R}}_\theta$ on a batch B . We decompose the expected error into sampling variance and model bias:

$$\text{Error} = \underbrace{\mathbb{E} \left[\|\widehat{\nabla_\phi J}(\phi) - \mathbb{E}_B[\widehat{\nabla_\phi J}(\phi)]\| \right]}_{\text{Sampling Variance}} + \underbrace{\|\mathbb{E}_B[\widehat{\nabla_\phi J}(\phi)] - \nabla_\phi J(\phi)\|}_{\text{Model Bias}}. \quad (\text{C.3})$$

The Sampling Variance term scales as $O(1/\sqrt{|B|})$ by the Central Limit Theorem applied to the batch sampling. The Model Bias term arises from substituting $\hat{\mathcal{R}}_\theta$ for $\mathcal{R}$. By the Lipschitz continuity of the policy gradient $\nabla_\phi \pi_\phi$ and the reward function gradients (Lemma 1), the difference in gradients is bounded by the uniform norm of the reward difference: $\|\nabla_\phi \hat{J} - \nabla_\phi J\| \leq L_\pi \|\hat{\mathcal{R}}_\theta - \mathcal{R}\|_\infty$. By Lemma 5, $\|\hat{\mathcal{R}}_\theta - \mathcal{R}\|_\infty \rightarrow 0$ as $|\mathcal{D}_i| \rightarrow \infty$ under the coverage guarantees of Lemma 4. Thus, the total error is bounded by the sum of the variance and the vanishing bias. $\square$

Theorem 3 (Stability of Iterative Deployment). *The iterative sequence of policies $\{\pi_{\phi_i}\}$ generated by Algorithm 2 converges to a stationary point of the true robust objective $J(\phi)$, and the system is stable against model collapse.*

Proof. We analyze the iterative update $\phi_{i+1} = \phi_i + \lambda \widehat{\nabla_{\phi} J}(\phi_i)$ where the objective J at iteration i utilizes the reward model $\hat{\mathcal{R}}_{\theta}^{(i)}$ trained on $\mathcal{D}_i$. Let $J_i(\phi) = \mathbb{E}_{x \sim \rho_x}[(1-\alpha)\hat{\mathcal{R}}_{\theta}^{(i)}(x, \pi_{\phi}(x)) + \alpha\eta(x, \pi_{\phi}(x))]$ denote the approximate objective at iteration i . From Lemma 2 and Theorem 2, the inner loop policy optimization (Algorithm 2, Lines 4-8) converges to a stationary point ϕ_i^* of $J_i(\phi)$ (within a neighborhood dependent on step size λ). From Lemma 6, the gradient error $\epsilon_i \rightarrow 0$ as $i \rightarrow \infty$ due to the convergence of $\hat{\mathcal{R}}_{\theta}^{(i)} \rightarrow \mathcal{R}$ (Lemma 5) and sampling variance $\rightarrow 0$ as $|B_i| \propto |D_i|$ and $|D_i| \rightarrow \infty$. Consequently, the sequence of objectives $J_i(\phi)$ converges uniformly to the true objective $J(\phi)$ over the compact parameter space Φ . Since the sequence of stationary points $\{\phi_i^*\}$ corresponds to a sequence of objectives converging uniformly to $J(\phi)$, the limit policy $\phi^* = \lim_{i \rightarrow \infty} \phi_i^*$ must satisfy $\nabla_{\phi} J(\phi^*) = 0$ (assuming isolated stationary points). The stability against model collapse is ensured because Lemma 3 and Lemma 4 prevent the support of $\mathcal{D}_i$ from collapsing, thereby maintaining the validity of the reward model estimation across the action space. $\square$

BIBLIOGRAPHY

- Asif, U., Tang, J., & Harrer, S. (2019). Densely supervised grasp detector (dsgd). *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01), 8085–8093.
- Bagnell, J. A., Ng, A. Y., & Schneider, J. G. (2001). Solving uncertain markov decision processes.
- Bellman, R. (1957). A markovian decision process. *Journal of mathematics and mechanics*, 6(5), 679–684.
- Billard, A. G., Calinon, S., & Guenter, F. (2006). Discriminative and adaptive imitation in uni-manual and bi-manual tasks. *Robotics and Autonomous Systems*, 54(5), 370–384.
- Calzada-Garcia, A., Victores, J. G., Naranjo-Campos, F. J., & Balaguer, C. (2025). A review on inverse kinematics, control and planning for robotic manipulators with and without obstacles via deep neural networks. *Algorithms*, 18(1), 23.
- Cunningham, P., Cord, M., & Delany, S. J. (2008). Supervised learning. In *Machine learning techniques for multimedia: Case studies on organization and retrieval* (pp. 21–49). Springer.
- Detry, R., Kraft, D., Kroemer, O., Bodenhagen, L., Peters, J., Krüger, N., & Piater, J. (2011). Learning grasp affordance densities. *Paladyn*, 2(1), 1–17.
- Feinberg, V., Wan, A., Stoica, I., Jordan, M. I., Gonzalez, J. E., & Levine, S. (2018). Model-based value expansion for efficient model-free reinforcement learning. *Proceedings of the 35th International Conference on Machine Learning (ICML 2018)*.
- Fujimoto, S., Meger, D., & Precup, D. (2019). Off-policy deep reinforcement learning without exploration. *International conference on machine learning*, 2052–2062.
- Gong, Z., Zhong, P., & Hu, W. (2019). Diversity in machine learning. *Ieee Access*, 7, 64323–64350.

- Greenacre, M., Groenen, P. J., Hastie, T., d’Enza, A. I., Markos, A., & Tuzhilina, E. (2022). Principal component analysis. *Nature Reviews Methods Primers*, 2(1), 100.
- Gurney, K. (2018). *An introduction to neural networks*. CRC press.
- Haarnoja, T., Tang, H., Abbeel, P., & Levine, S. (2017). Reinforcement learning with deep energy-based policies. *International conference on machine learning*, 1352–1361.
- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *International conference on machine learning*, 1861–1870.
- Han, D., Mulyana, B., Stankovic, V., & Cheng, S. (2023). A survey on deep reinforcement learning algorithms for robotic manipulation. *Sensors*, 23(7), 3762.
- Hussein, A., Gaber, M. M., Elyan, E., & Jayne, C. (2017). Imitation learning: A survey of learning methods. *ACM Computing Surveys (CSUR)*, 50(2), 1–35.
- Jaiswal, A., Babu, A. R., Zadeh, M. Z., Banerjee, D., & Makedon, F. (2020). A survey on contrastive self-supervised learning. *Technologies*, 9(1), 2.
- Kumar, A., Zhou, A., Tucker, G., & Levine, S. (2020). Conservative q-learning for offline reinforcement learning. *Advances in neural information processing systems*, 33, 1179–1191.
- Langford, J., & Zhang, T. (2007). The epoch-greedy algorithm for multi-armed bandits with side information. *Advances in neural information processing systems*, 20.
- Levine, S., Pastor, P., Krizhevsky, A., Ibarz, J., & Quillen, D. (2018). Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International journal of robotics research*, 37(4-5), 421–436.
- Li, C., Liu, K., & Liu, S. (2025). A survey of loss functions in deep learning. *Mathematics*, 13(15), 2417.
- Li, Y. (2017). Deep reinforcement learning: An overview. *arXiv preprint arXiv:1701.07274*.
- Liu, Q., Kuang, Y., & Wang, J. (2024). Robust deep reinforcement learning with adaptive adversarial perturbations in action space. *2024 International Joint Conference on Neural Networks (IJCNN)*, 1–8.

- Luo, F.-M., Xu, T., Lai, H., Chen, X.-H., Zhang, W., & Yu, Y. (2024). A survey on model-based reinforcement learning. *Science China Information Sciences*, 67(2), 121101.
- Mahler, J., Liang, J., Niyaz, S., Laskey, M., Doan, R., Liu, X., Ojea, J. A., & Goldberg, K. (2017). Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics. *arXiv preprint arXiv:1703.09312*.
- Mao, A., Mohri, M., & Zhong, Y. (2023). Cross-entropy loss functions: Theoretical analysis and applications. *International conference on Machine learning*, 23803–23828.
- Matsushima, T., Furuta, H., Matsuo, Y., Nachum, O., & Gu, S. (2020). Deployment-efficient reinforcement learning via model-based offline optimization. *arXiv preprint arXiv:2006.03647*.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *nature*, 518(7540), 529–533.
- Moos, J., Hansel, K., Abdulsamad, H., Stark, S., Clever, D., & Peters, J. (2022). Robust reinforcement learning: A review of foundations and recent advances. *Machine Learning and Knowledge Extraction*, 4(1), 276–315.
- Nachum, O., Dai, B., Kostrikov, I., Chow, Y., Li, L., & Schuurmans, D. (2019). Algaedice: Policy gradient from arbitrary experience. *arXiv preprint arXiv:1912.02074*.
- Nagabandi, A., Kahn, G., Fearing, R. S., & Levine, S. (2018). Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. *2018 IEEE international conference on robotics and automation (ICRA)*, 7559–7566.
- Parker, A. (2025, February). *Buffalo Gym* (Version 0.4.0). <https://github.com/foreverska/buffalo-gym>
- Pinto, L., & Gupta, A. (2016). Supersizing self-supervision: Learning to grasp from 50k tries and 700 robot hours. *2016 IEEE international conference on robotics and automation (ICRA)*, 3406–3413.
- Precup, D., Sutton, R. S., & Singh, S. (2000). Eligibility traces for off-policy policy evaluation.

- Prudencio, R. F., Maximo, M. R., & Colombini, E. L. (2023). A survey on offline reinforcement learning: Taxonomy, review, and open problems. *IEEE transactions on neural networks and learning systems*, 35(8), 10237–10257.
- Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic policy gradient algorithms. *International conference on machine learning*, 387–395.
- Slivkins, A. (2024). Introduction to multi-armed bandits. <https://arxiv.org/abs/1904.07272>
- Smith, L., Kew, J. C., Peng, X. B., Ha, S., Tan, J., & Levine, S. (2022). Legged robots that keep on learning: Fine-tuning locomotion policies in the real world. *2022 international conference on robotics and automation (ICRA)*, 1593–1599.
- Sutton, R. S. (1991). Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4), 160–163.
- Sutton, R. S., Barto, A. G., et al. (1998). *Reinforcement learning: An introduction* (Vol. 1). MIT press Cambridge.
- Tan, K. L., Esfandiari, Y., Lee, X. Y., Sarkar, S., et al. (2020). Robustifying reinforcement learning agents via action space adversarial training. *2020 American control conference (ACC)*, 3959–3964.
- Tessler, C., Efroni, Y., & Mannor, S. (2019). Action robust reinforcement learning and applications in continuous control. *International Conference on Machine Learning*, 6215–6224.
- Vecerik, M., Hester, T., Scholz, J., Wang, F., Pietquin, O., Piot, B., Heess, N., Rothörl, T., Lampe, T., & Riedmiller, M. (2017). Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards. *arXiv preprint arXiv:1707.08817*.
- Wu, P., Escontrela, A., Hafner, D., Abbeel, P., & Goldberg, K. (2023). Daydreamer: World models for physical robot learning. *Conference on robot learning*, 2226–2240.
- Xu, H., & Mannor, S. (2006). The robustness-performance tradeoff in markov decision processes. *Advances in Neural Information Processing Systems*, 19.

Yu, T., Thomas, G., Yu, L., Ermon, S., Zou, J. Y., Levine, S., Finn, C., & Ma, T. (2020). Mopo: Model-based offline policy optimization. *Advances in neural information processing systems*, 33, 14129–14142.